

Attacks on Vision-Language-Action Robotic Systems: Mechanisms, Evaluation, and Open Problems

Anonymous Author(s)

Abstract—With the rapid advancement of Vision-Language-Action (VLA) models, ensuring the security of embodied intelligent systems has emerged as a critical area of research. This survey provides a comprehensive analysis of VLA safety, exploring the unique vulnerabilities that arise when multimodal reasoning is directly coupled with physical execution. We introduce a unified, mechanism-driven framework that integrates the entire attack lifecycle, offering a holistic perspective on how adversarial interventions manipulate perception, planning, and control. Through a systematic analysis of VLA architectures, we establish a classification framework that distinguishes direct embodied threats from traditional digital attacks, providing standardized criteria for evaluating attack success in closed-loop environments. We identify critical limitations in existing research regarding physical evaluations, privacy metrics, and adaptive safeguards, and we outline future directions to strengthen VLA robustness. Our findings offer strategic recommendations for advancing embodied security and ensuring the reliable deployment of robotic foundation models in high-stakes, real-world applications. To support ongoing research in this field, we maintain a public project page continuously updating the latest work on VLA attack safety: <https://xtli12.github.io/VLA-Attack-Survey/>.

Index Terms—Vision-Language-Action Attacks, Robot Foundation Models, Survey, Embodied Security, Physical Attack Evaluation.

I. INTRODUCTION

Vision-Language-Action (VLA) models and robot foundation models are reshaping the interface between perception, language, and control. Rather than classify an image or emit a low-level action for a fixed task, a VLA system uses open-ended language and visual context to condition robot action, memory, tool use, and future observations. Early embodied and VLA systems connect multimodal pretraining to robot action and language-conditioned manipulation [1]–[4]. Large robot-data efforts and open policies extend this shift across embodiments and datasets [5]–[8]. Recent generalist and adapted VLA/RFM systems further emphasize video pretraining, action tokenization, humanoid control, and lightweight action adapters [9]–[12] [13]–[15]. Related planner and policy systems expose the same pattern at different interfaces: language and vision can steer skill selection, code generation, waypoint synthesis, trajectory distributions, and action-chunk prediction [16]–[19] [20]–[24]. For robotics, this is more than a new model family. It is a control interface whose inputs are semantically rich and whose outputs can move robots, call tools, update memory, and alter the future observation stream.

This interface changes what an attack means. In a VLM, an adversarial image may produce a wrong answer; in a

text-only LLM, a jailbreak may produce unsafe text; in a conventional robot policy, a disturbance may reduce reward or tracking accuracy [25]–[29]. In a VLA-enabled robot, similar interventions can enter a closed loop. A visual patch can change grounding; the changed grounding can alter an action token or waypoint; the controller may clip or smooth the command; and the next observation may either recover the task or reinforce the deviation. A scene-text instruction can cross a source boundary, a poisoned demonstration can implant a trigger, a black-box prompt edit can redirect a trajectory, and an action-level backdoor can freeze or release a gripper at a critical moment. The consequence is not only a misclassification or unsafe sentence, but potentially a wrong-object action, unsafe motion, unauthorized skill use, privacy leakage, or persistent state compromise.

A. Related Surveys and Positioning

Existing survey traditions only partly cover this closed-loop attack problem. VLA/RFM surveys emphasize model architectures, datasets, action representations, and applications. Embodied-security and robot-cybersecurity surveys cover sensors, middleware, physical platforms, and human-robot interaction. LLM-agent security surveys address prompt injection, tool misuse, retrieval poisoning, and data leakage. The closest adjacent work is the VLA safety survey *Vision-Language-Action Safety: Threats, Challenges, Evaluations, and Mechanisms* [30]–[32], which covers the broader safety landscape: threats, alignment, evaluation, defense mechanisms, and deployment challenges. This paper asks a narrower attack-centered question: *how do adversarial interventions propagate through VLA-controlled robots, what mechanisms have been demonstrated, and when is the evidence strong enough to support a robotics claim?*

This distinction has become practical as the literature moves from position statements to concrete VLA attack papers. Direct evidence now includes adversarial prompt and patch attacks on VLA policies [33]–[36]. Action-level and backdoor work targets freezing, primitives, physical-object triggers, long-horizon behavior, and chunk drift [37]–[40] [41], [42]. Other direct studies examine transferable patches, 3D object textures, CoT-reasoning hijacking, trajectory redirection, and membership inference [43]–[46] [47]–[49]. These papers are not merely examples within a safety taxonomy; they are the attack evidence this survey compares.

TABLE I
OVERVIEW OF RELATED SURVEY TRADITIONS AND HOW THIS SURVEY IS POSITIONED. THE COMPARISON CONCERNS SCOPE AND EVIDENCE USE, NOT THE VALUE OF EACH LITERATURE.

Adjacent literature	Direct VLA attack corpus	Action/control representation	Controller/recovery boundary	Evidence calibration	Gap addressed here
VLA/RFM surveys [50], [51]	×	✓	<i>Partial</i>	<i>Partial</i>	Explain model architectures, datasets, action heads, and applications, but usually treat attacks as peripheral robustness or safety issues.
VLA safety survey [30]	<i>Partial</i>	<i>Partial</i>	<i>Partial</i>	✓	Covers threats, alignment, defenses, evaluations, and deployment challenges across VLA safety; it is broader than an attack-mechanism and evidence-strength analysis.
Embodied and robot security surveys [52], [53]	×	×	✓	<i>Partial</i>	Explain sensors, middleware, cyber-physical vulnerabilities, HRI, and deployed robot risks, but do not center VLA policies or VLA action representations.
LLM-agent and tool-risk studies [54]–[56]	×	×	<i>Partial</i>	✓	Clarify prompt injection, source confusion, tool misuse, and benchmark design, but software-agent success does not establish robot-level VLA consequences.
Adversarial ML and physical-attack work [57]–[60]	×	×	<i>Partial</i>	✓	Provides mechanisms for perturbations, transfer, and physical realization, but sensor failure must still be connected to VLA grounding, action, execution, and recovery.
This paper	✓	✓	✓	✓	Centers attacks on VLA-controlled robots, separates direct VLA evidence from adjacent work, and evaluates claims through action representation, controller boundary, validation tier, privacy, and recovery.

B. Guiding Questions

We use five questions to keep the survey focused on mechanisms and robotic consequences.

- **RQ1:** Where do VLA attacks enter the robot system: vision, language, memory, tools, environment, action representations, or humans?
- **RQ2:** Which internal objects do attacks change: grounding, plans, reasoning traces, action tokens, chunks, trajectories, controller inputs, memory entries, or tool calls?
- **RQ3:** How do these changes propagate through execution, feedback, recovery, and future observations?
- **RQ4:** Which consequences have been demonstrated: task failure, wrong-object action, unsafe motion, privacy leakage, unauthorized skill use, or persistent compromise?
- **RQ5:** What evidence is strong enough for robotics conclusions, and where do physical validation, transfer, controller interaction, and recovery remain weak?

Table I compares this survey with the closest adjacent survey traditions. The difference is not simply that we are “attack-centered.” The central issue is how attack mechanisms interact with VLA action representations, closed-loop execution, controller boundaries, and the strength of the available evidence.

C. Scope and Definition

We treat a *VLA attack* as an intentional intervention that changes a VLA-enabled system’s perception, grounding, planning, policy decoding, execution, memory, tool use, privacy boundary, or human-robot interaction in a way that violates the intended task, authorization boundary, privacy expectation, or safety envelope. This separates adversarial attacks from

ordinary task failure and non-adversarial robustness failure. A robot that fails because of unmodeled friction exhibits robustness risk; a robot that fails because an adversary selected a perturbation, instruction, tool response, memory entry, trigger, or physical scene configuration to induce a violation exhibits security risk.

D. Evidence Use

We use a compact evidence review to calibrate claims, while organizing the survey around attack mechanisms and robotic consequences. Searches were last updated on 16 June 2026 and covered major robotics, machine learning, computer vision, security, and NLP venues, along with arXiv/OpenReview and linked project pages.

The survey distinguishes direct VLA evidence from embodied-adjacent and contextual work. *Direct VLA evidence* requires a VLA target and a VLA-relevant outcome, such as changed action, trajectory, reasoning-to-action behavior, triggered policy behavior, or training-data membership leakage. *Embodied-adjacent evidence* motivates risks around embodied planners, robot policies, sensing paths, middleware, and HRI, but it still requires VLA-specific validation before it can support a VLA vulnerability claim. *Contextual evidence* explains substrates, benchmarks, adversarial methods, robot-security mechanisms, or action representations. Supplementary materials provide the search templates and evidence appendix. The main text uses this evidence base to support mechanism-level conclusions, not to pool heterogeneous ASR values.

E. Contributions

This paper discusses attacks at a responsible academic level and does not provide operational payloads, exploit recipes, or

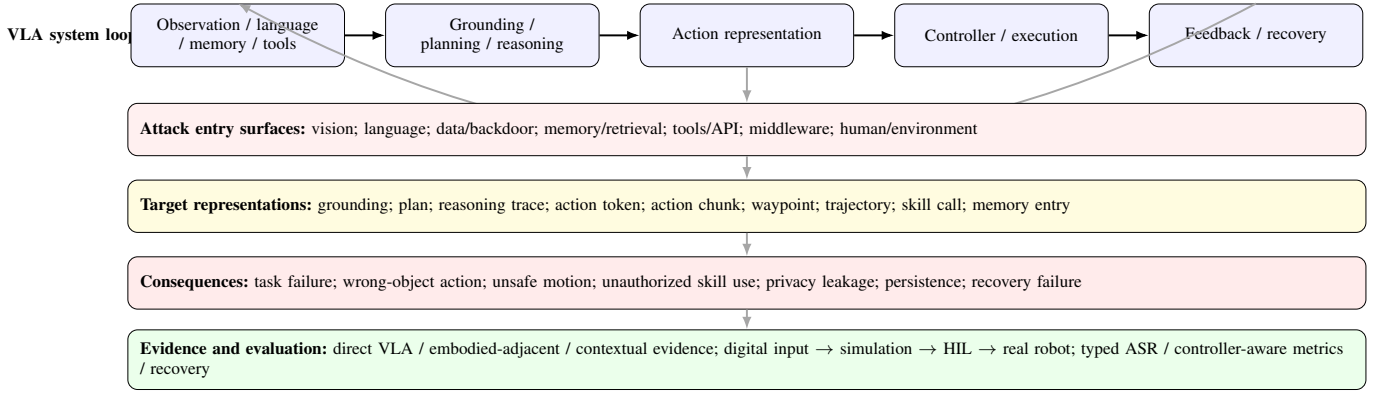


Fig. 1. VLA attack survey overview. The figure summarizes the scope of the survey by linking the VLA system loop to attack entry surfaces, target representations, consequences, and evidence/evaluation levels.

instructions for real-world misuse. It makes six contributions.

- We define the VLA attack problem as a closed-loop robotics security problem, where adversarial effects can propagate from perception, language, data, memory, tools, or middleware into robot actions, physical state, and recovery behavior.
- We formalize typed VLA attack success, separating task failure, targeted behavior, safety violation, authorization violation, privacy leakage, persistence, and recovery failure.
- We introduce an orthogonal attack map covering lifecycle, entry surface, target component, target representation, security property, attacker assumptions, validation tier, and consequence.
- We synthesize direct VLA attacks through their robotic mechanisms, with particular attention to action tokens, chunks, waypoints, trajectories, diffusion/flow policies, controller clipping, and recovery.
- We compare the strength and limits of current evidence, separating direct VLA attacks from embodied-adjacent attacks and contextual references without letting background work inflate attack claims.
- We propose evaluation principles for VLA attacks that connect typed ASR, benign utility, transfer, query budget, physical validation, closed-loop recovery, and consequence severity.

F. Survey Overview

Fig. 1 provides an overview of the survey. The argument proceeds from the robot loop to attack mechanisms, evidence strength, evaluation, and open problems. Section II first defines how adversarial effects enter through an entry surface, change a control-relevant representation, cross the controller boundary, and end in recovery or violation. Section III then asks how existing attacks instantiate this propagation pattern. Section IV separates direct VLA evidence from embodied-adjacent and contextual evidence. Section V turns this evidence into typed attack claims, and Section VI identifies what remains missing for deployment-relevant VLA attack conclusions.

II. CLOSED-LOOP PROPAGATION OF VLA ATTACKS

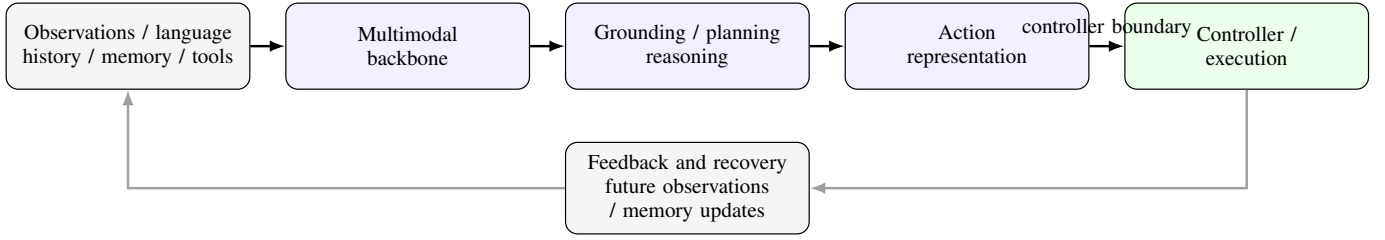
We begin by making the robot loop explicit, because the rest of the survey treats a VLA attack as a propagation process: an entry surface changes a representation, the change may cross the controller boundary, and feedback either supports recovery or carries the system toward violation.

A. VLA Systems as Semantic-to-Physical Loops

Vision-Language-Action (VLA) models extend multimodal foundation models from interpretation to embodied control. Given scene observations, a language goal, and often a short interaction history, they produce robot-relevant actions or action-conditioned decisions. Their defining feature is not simply that they accept images and text. It is that semantic understanding is coupled to behavior: object names, spatial relations, affordances, task constraints, and user intent are converted into motions, skill calls, or trajectories that change the world. VLA systems therefore continue the lines of vision-language modeling and robot imitation learning, while making failures more consequential than errors in captioning or static recognition [1], [3], [6]–[8] [50], [51], [61].

This control role separates VLA systems from adjacent model classes. A VLM may ground objects, answer questions, or describe a scene, but its output need not be executable. An LLM agent may call software tools, search memory, or follow a procedure, but it may have no direct contact with robot dynamics. A conventional robot policy controls motion, but it is usually conditioned on a narrow task representation rather than open-ended language and web-scale semantics. A VLA system sits at the intersection: it binds perception and language to actions inside a feedback loop, so an error in grounding, reasoning, or instruction interpretation can become a changed grasp, path, tool invocation, or recovery decision.

A typical VLA pipeline runs from context to control. The robot observes the world through cameras, proprioception, and other sensors; receives a language goal from a user or task manager; and conditions on recent history, memory, retrieved context, or prior actions when available. A multi-modal backbone maps this context into representations for object grounding, spatial reasoning, affordance estimation, or task decomposition. A planner, reasoner, or policy head then



A VLA system couples semantic context to executable behavior; actions change the future observations, memory, tool state, and recovery choices that condition later decisions.

Fig. 2. VLA system loop used as background for the attack analysis. The figure abstracts the deployed robot stack from multimodal context through grounding/planning, action representation, controller execution, and feedback/recovery.

chooses an action representation: a token, an action chunk, a waypoint, a trajectory, a sampled action sequence, or a skill/API call. Finally, a controller, motion planner, or runtime executor turns that representation into actuation, after which the new physical state produces the next observation. Fig. 2 summarizes this system view. For this survey, the key point is that the pipeline is not a one-way perception module followed by a separate robot controller. It is a semantic-to-physical loop [62]–[66].

B. Action Representations and Attack Surfaces

VLA architectures differ most visibly in how they represent the step from semantic context to executable behavior. End-to-end policies map observations and language directly to actions or action distributions; RT-style models, OpenVLA, and Octo illustrate this interface for robot-data-scale policies [2], [3], [6], [7]. Other generalist policies extend the same idea through multi-embodiment training, video pretraining, or humanoid-oriented foundation models [9], [10], [12], [14]. VLM-to-policy adapters and native tokenized VLA formulations further expose the perception-to-action bridge as a design object [11], [13], [15]. Planner-skill systems use a VLM or LLM to select skills, synthesize code, build value maps, or rank affordances while lower-level policies execute the selected behavior [16]–[19]. Language-feedback, preference, and action-hierarchy systems show related planner/skill interfaces for embodied agents [20], [63]–[65]. Action-token models discretize motor commands and inherit the behavior of autoregressive decoding and detokenization [2], [3], [6]. Spatial action grids and compressed action tokenizers create more specialized token-to-control mappings [23], [67]. Chunked policies emit several future actions per inference step [22], [68]. FAST-style tokenization and diffusion transformers also make the action chunk itself a design unit [23], [24]. Waypoint and trajectory policies expose geometric targets and path-level objectives. Transporter-style and CLIPort policies use spatial action targets [20], [69], while PerAct, Act3D, and RVT predict 3D action maps or gripper poses [70]–[72]. Diffusion and flow policies generate conditional action sequences through sampling or velocity fields rather than a single deterministic output [8], [21], [73]. 3D diffusion, diffusion-transformer, and hierarchical diffusion policies make the trajectory distribution, denoising process, or sampler part of the action interface [24], [74]–[76]. Skill/API agents make

the action more symbolic by selecting tools, services, robot skills, or arguments that execute outside the model.

These representations are not merely implementation details; they define attack targets and evaluation signals. An action-token attack may be measured by changed token probability, wrong gripper state, or an autoregressive error that propagates into later commands [23], [77], [78]. A chunk-level attack may be small at each step but consequential because feedback is delayed until the chunk is partly executed [22], [42], [68]. A waypoint attack can be evaluated by wrong-object grounding, endpoint displacement, or unsafe intermediate poses; a trajectory attack can target curvature, timing, clearance, or path redirection. Diffusion and flow policies expose the conditional distribution, denoising path, velocity field, seed, or guidance mechanism, so the relevant object may be a sampled trajectory rather than a single decoded action. Skill/API calls shift the concern toward authorization, source confusion, and argument integrity. This is why later sections treat “action” as a family of control-relevant representations rather than one uniform output type.

Because these representations sit inside a deployed loop, attacks can enter through perception, language, memory, retrieved context, training data, tool output, controller interfaces, the physical environment, or human interaction. The key question is how an intervention moves from a semantic change, such as altered grounding or planning, to a control change, such as a token, chunk, waypoint, trajectory, diffusion sample, or skill call. The closed-loop model below gives the compact formal view used throughout the survey.

C. Closed-Loop Attack Model

With this background in view, a VLA-enabled robot is better read as a closed-loop state transition system than as a single prompt-to-action function. Let x_t denote the physical state, $o_t = O(x_t, \eta_t)$ the observation produced by sensors and preprocessing noise η_t , m_t the memory/tool/human context, and u_t the command emitted by the VLA or its planner. A low-level controller K maps u_t and robot state to actuation, while a recovery or safety mechanism R may stop, replan, filter, or ask for help [79]–[82]. We use the following compact

state-update view:

$$\begin{aligned} u_t &= \pi_\theta(o_{\leq t}, g, m_t), \\ a_t &= K(u_t, x_t, R_t), \\ x_{t+1} &= F(x_t, a_t, w_t), \\ m_{t+1} &= U(m_t, o_t, u_t, a_t). \end{aligned} \quad (1)$$

The safety envelope is a set S over physical state, controller state, tool authorization, privacy state, and human proximity. An attack is relevant when an intervention δ changes one of these transitions or contexts so that the rollout violates a task, authorization, privacy, or safety predicate. This view gives the survey its central question: where does the adversary enter, which representation changes first, how does the change cross the controller boundary, and does feedback or recovery stop it?

At this level, adversarial propagation can be summarized as

$$\begin{aligned} \delta &\rightarrow \Delta o_t \rightarrow \Delta z_t \rightarrow \Delta u_t \rightarrow \Delta a_t \\ &\rightarrow \Delta x_{t+1} \rightarrow \text{recovery or violation} \end{aligned} \quad (2)$$

Fig. 1 situates this propagation chain within the broader survey scope: entry surfaces, target representations, consequences, and evidence levels. The same perturbation can stop at the observation, change an embedding or grounding representation, alter a plan, shift an action token, corrupt a chunk, redirect a trajectory, be clipped by a controller, or appear only after the next observation. A VLA attack claim is therefore not settled at the model-output boundary; the robot-level claim depends on execution, feedback, recovery, and the resulting physical or system consequence.

D. Attack Map

The attack map has four intuitive stages. First, the attack enters through a channel: vision, language, memory, tool output, middleware, training data, physical environment, or human interaction. Second, it changes a target representation: grounding, plan, reasoning trace, action token, action chunk, waypoint, trajectory, velocity field, controller state, memory entry, or tool call. Third, the changed representation reaches a robot or system consequence, such as task failure, wrong-object action, unsafe motion, unauthorized skill use, privacy leakage, or persistent state compromise. Fourth, the validation setting determines what the claim can support: digital input, real-sensor input, simulation rollout, hardware-in-the-loop, real-robot actuation, or human-proximate study.

Table II keeps these stages orthogonal because familiar labels often hide different mechanisms. “Physical attack” describes an access channel, not the target: a physical patch may attack visual embeddings, grounding, visual proprioception, or a trajectory objective. “Jailbreak” describes a family resemblance, not a consequence: a prompt attack may cause task failure, targeted motion, unauthorized tool use, or no robot-level effect. “Backdoor” describes timing and training access, but the robotic target may be a gripper primitive, an action chunk, a waypoint, or a long-horizon trajectory.

E. Typed Attack Success

Because VLA safety papers do not yet use a uniform definition of attack success, the survey needs a typed definition. Let $\tau^0(c)$ be the benign rollout from an initial condition and task context $c = (x_0, g, m_0)$, and let $\tau^\delta(c)$ be the rollout under an adversarial intervention δ that satisfies the declared knowledge, access, privilege, budget, and physical constraints. A VLA attack is successful for objective type O and consequence class C when

$$\begin{aligned} \text{Succ}_{O,C}(\tau^\delta, \tau^0) &= \mathbf{1}\{\phi_{O,C}(\tau^\delta) = 1 \\ &\quad \wedge \phi_{O,C}(\tau^0) = 0 \wedge \delta \in \Delta(\mathcal{A})\}. \end{aligned} \quad (3)$$

where $\phi_{O,C}$ is a typed violation predicate and $\Delta(\mathcal{A})$ is the admissible attacker set. The benign rollout condition prevents ordinary task difficulty or non-adversarial distribution shift from being counted as attack success. If a benign rollout is unavailable, the paper should state the baseline used, such as clean task success, human-labeled intended behavior, or non-member trajectory statistics.

The predicate $\phi_{O,C}$ must be typed. A *task-failure attack* succeeds when the benign task would complete but the attacked rollout fails. A *targeted behavior attack* succeeds when the robot reaches an attacker-specified object, action primitive, waypoint, trajectory, skill, or tool call. A *safety attack* succeeds when the rollout violates a safety envelope, contact margin, force/velocity/workspace bound, or human-proximity constraint. An *authorization attack* succeeds when the system executes an unauthorized command, skill, tool, or privilege transition. A *privacy attack* succeeds when membership, trajectory, user, or scene information is inferred above a stated threshold. A *persistence attack* succeeds when the adversarial effect survives replanning, feedback, memory update, fine-tuning, or a session boundary. A *recovery-failure attack* succeeds when the system has an opportunity to stop, replan, ask for help, or invoke a recovery policy but still reaches the violation predicate.

This definition makes ASR meaningful only when five items are stated together: the violation predicate, the denominator, the attacker assumptions, the validation tier, and the benign or clean baseline. Without them, ASR values from different VLA papers should be treated as local evidence rather than comparable field-level rates.

F. Attacker Assumptions

We assume the deployer has an intended task, authorized users, authorized tools and skills, and a safety envelope for people, property, data, and robot hardware. The attacker seeks to violate one or more of these constraints while preserving enough surface plausibility for the system to keep acting, log the event as normal, or store corrupted context for future sessions.

Attacker capability has three independent dimensions. *Knowledge level* describes what the attacker knows about weights, gradients, architecture, prompts, action heads, or controllers: white-box, gray-box, black-box, transfer-only, or unknown. *Access channel* describes how the intervention is delivered: digital, physical, human-mediated, supply-chain,

TABLE II
ORTHOGONAL ATTACK-MAP DIMENSIONS FOR VLA ATTACKS. EACH PAPER MAY TOUCH MULTIPLE DIMENSIONS; THE DIMENSIONS ARE NOT MUTUALLY EXCLUSIVE CATEGORIES.

Axis	Common labels	Guiding question
Lifecycle	Training; inference; runtime; post-deployment	When is the adversarial intervention introduced or activated?
Entry surface	Vision; language; state; memory; tool; middleware; human; environment	Which semantic or system channel carries the adversarial content?
Target component	Visual encoder; language encoder; multimodal projector; planner/reasoner; action head; controller; memory; tool/API; middleware	Which system component is attacked or measured?
Target representation	Embedding; grounding; plan; action token; chunk; waypoint; trajectory; velocity field; controller state; memory entry	Which representation or control object changes inside or across components?
Security property violated	Availability; integrity; confidentiality; authorization	Which security property is violated? Physical safety is represented as a consequence rather than merged into this axis.
Knowledge level	White-box; gray-box; black-box; transfer-only; unknown	What does the attacker know about weights, gradients, architecture, prompts, action heads, or controllers?
Access channel	Digital; physical; human-mediated; supply-chain; hybrid	How does the attacker deliver the intervention? This is separate from knowledge level.
Privilege level	User; bystander; operator; data contributor; model provider; tool/API; middleware; no privilege	What role or privilege boundary does the attacker occupy or abuse?
Temporal profile	One-shot; repeated; delayed; persistent	Does the attack act once, recur over feedback, activate later, or survive sessions/training?
Validation tier	Digital input; real sensor; simulation; HIL; robot actuation; human-proximate	What is the strongest demonstrated validation tier? Do not infer actuation or human proximity from input-only tests.
Consequence	Task failure; wrong object; unsafe action; collision/contact risk; privacy leakage; unauthorized skill; persistent state compromise	What happened at the robot/system boundary? Keep consequence separate from objective and entry surface.

or hybrid. *Privilege level* describes the role or boundary involved: user, bystander, operator, data contributor, model provider, tool/API, middleware, or no privilege. Keeping these dimensions separate avoids a common confusion: “black-box” and “physical” are not alternatives. A physical patch can be optimized with white-box gradients, transferred in a black-box setting, or deployed by a bystander with no digital privileges.

G. Reading the Map Through Examples

The map is easiest to read through examples. In a scene-text hijacking scenario, the attacker need not rewrite the user’s goal: the robot sees an instruction-like phrase in the environment, the phrase changes source grounding or plan selection, and the downstream consequence may be wrong-object manipulation or unauthorized skill use. In a patch scenario, the entry is visual, but the relevant question is whether the perturbation merely changes an embedding or whether it reaches an action token, waypoint, or physical trajectory [34], [43], [44]. In a poisoned-demonstration or model-supply scenario, the attack enters before deployment, hides behind clean utility, and later activates as a gripper primitive, action freeze, initial-state trigger, persistent module-level backdoor, chunk-level drift, or flow-vector-field hijack [37], [40], [42], [83]–[85]. In a trajectory-redirection scenario, the language may remain close to the user’s instruction while the physical path bends toward a different endpoint [47]. In a membership-inference scenario, the consequence is not motion failure at all; action likelihoods or temporal traces leak whether a demonstration or trajectory was in the training set [48], [49].

This separation also prevents overclaiming. A VLM jailbreak with no robot action is contextual evidence for source confusion, not direct evidence of unsafe VLA action. A ROS2 middleware exploit is embodied-adjacent evidence for tool or

middleware compromise, not a VLA policy attack unless the VLA system consumes or emits the compromised message. A physical LiDAR or camera attack can be strong sensor evidence, but it remains contextual for VLA action unless a VLA-controlled robot rollout is measured. With the loop and typed claim structure in place, the next section asks how existing attacks actually enter the loop and which representations they change.

III. EVOLUTION OF VLA ATTACK MECHANISMS

With the loop in place, we next ask how attacks enter it and what they change before any robot-level consequence appears. Building on the orthogonal map in Section II, this section groups concrete VLA attacks by their primary mechanism and target representation. The groups are not mutually exclusive: a backdoor may also be action-level, physical, persistent, or dynamics-aware.

The field has developed in waves. The first grew out of adversarial vision and physical patches: if camera inputs can be perturbed, robot perception and grounding may change. The second drew on prompt injection, embodied jailbreaks, and robot-policy attacks, where language and task context steer plans, skills, and sequential decisions. The current wave attacks VLA systems directly: patches are optimized for action deviation, prompts are evaluated by rollout behavior, backdoors target primitives and chunks, reasoning traces are treated as control-relevant state, trajectories are redirected, and action traces become privacy signals. This section follows that evolution by mechanism rather than by modality alone. Table III summarizes representative VLA attack methods and their evaluation settings.

TABLE III

REPRESENTATIVE VLA ATTACK METHODS AND EVALUATION SETTINGS. OBJECTIVES USE F = TASK FAILURE/AVAILABILITY, T = TARGETED BEHAVIOR, S = SAFETY VIOLATION, P = PRIVACY LEAKAGE, AND A = AUTHORIZATION MISUSE. VALIDATION USES D = DIGITAL INPUT, SIM = SIMULATION ROLLOUT, HIL = HARDWARE-IN-THE-LOOP, AND ROBOT = REAL ROBOT/WORLD ACTUATION. PR = PEER-REVIEWED; PP = PREPRINT OR NON-ARCHIVAL VERSION.

Work	Venue / status	Attack strategy	Obj.	Knowledge / access	Entry surface	Target representation	Validation	Victim model / benchmark
<i>Instruction and reasoning attacks</i>								
RoboGCG [33]	PP	Adversarial prompt optimization for control authority	F/T	Gray/black-box; digital	Language	Action reachability / rollout behavior	D, Sim	VLA policies / manipulation rollouts
SABER [36]	PP	Bounded black-box instruction edits	F/S	Black-box; query access	Language	Plan and action sequence	D, Sim	OpenVLA-style policies / LIBERO
Q-DIG [86]	PP	Quality-diversity red-team prompts	F	Black-box; digital	Language	Task plan / policy behavior	Sim	Robot-policy benchmarks
TRAP [46]	PP	Patch-induced CoT/reasoning hijack	T/S	Physical/digital patch	Vision + reasoning	CoT trace / entity grounding	D, Sim, Robot	Reasoning-enabled VLA manipulation
Trajectory redirection [47]	PP	Command-preserving prompt redirection	T/S	Digital prompt	Language	Trajectory / endpoint	Sim	VLA manipulation rollouts
<i>Visual and physical attacks</i>								
VLA adversarial patch [34]	PR	Action-space adversarial patch / trajectory loss	F/T/S	White/gray-box; patch	Vision	Grounding, action, trajectory	D, Sim, Robot	VLA policies / manipulation tasks
VLA-Fool [35]	PP	Textual, visual, and cross-modal misalignment	F/T	Digital input	Vision + language	Multimodal alignment / action	D, Sim	VLA models / manipulation suites
EDPA / ADVLA [43], [87]	PP	Embedding, attention, and projected-feature attacks	F/T	White/transfer; digital	Vision	Visual embedding / attention	D, Sim	OpenVLA-style targets
UPA-RFAS / VLA-Hijack [44], [88]	PR/PP	Transferable patch / visual proprioception hijack	F/T/S	Transfer / black-box patch	Vision	Visual-proprioceptive features	D, Sim, Robot	Multiple VLA policies
Tex3D [45]	PP	Adversarial 3D object texture	F/T	Physical object access	Environment / vision	Object texture / grounding	Sim, Robot	VLA manipulation tasks
Partial-observation patch [89]	PR	Patch optimized from short trajectory prefix	F/T	Partial-observation access	Vision	Trajectory-relevant visual state	Sim	VLA robot rollouts
<i>Backdoor and action-level attacks</i>								
FreezeVLA [37]	PP	Triggered action freezing	F/T	Training-time / trigger	Data + runtime trigger	Static action state / primitive	Sim	VLA manipulation policies
BadVLA [38]	PP	Objective-decoupled VLA backdoor	T	Training-time / trigger	Data / physical object	Goal or action behavior	Sim	VLA policies / manipulation suites
GoBA [39]	PP	Physical-object-triggered backdoor	T/S	Training-time + physical trigger	Physical object	Goal-conditioned behavior	Sim, Robot	VLA manipulation tasks
DropVLA / AttackVLA [40], [41]	PP	Action-level / long-horizon triggered behavior	F/T	Training-time / trigger	Data + action head	Primitive / sequence / action head	Sim	OpenVLA-style targets
INFUSE / State Backdoor [83], [84]	PP	Persistent module-level / state-space backdoors	T/S	Model-supply or training-time / trigger	Base model + robot state	Fine-tune-insensitive modules / initial robot state	Sim, Robot	Multiple VLA policies / real robot tasks
FlowHijack [85]	PR	Dynamics-aware flow-matching backdoor	T/S	Training-time / context trigger	Data + flow policy	Vector-field dynamics / continuous trajectory	Sim, Robot	Flow-matching VLA policies
SilentDrift [42]	PP	Stealthy chunk-level drift	F/T/S	Training-time / trigger	Data + action chunk	Delta-pose chunk / trajectory	Sim	Chunked VLA policies
<i>Privacy attacks</i>								
VLA membership inference [48]	PP	Membership inference from action likelihoods/errors	P	Query / output access	Model outputs	Action likelihood / error trace	D, Sim	VLA policies / trajectory data
VLALeaks [49]	PP	Membership inference from temporal action traces	P	Query / output access	Model outputs	Attention / temporal motion signature	D, Sim	VLA models / robot traces
<i>Embodied-adjacent attacks</i>								
BadRobot / RoboPAIR [90], [91]	PR/PP	Embodied jailbreak / harmful plan induction	F/S/A	Digital or interactive	Language	Plan / skill sequence	D, Sim, Robot	LLM-controlled robots
CHAI / RoboJailBench [92], [93]	PP	Command hijacking / embodied jailbreak benchmark	F/S/A	Digital + physical context	Language / environment	Plan, tool, or skill call	D, Sim	Embodied agents / robot planners
ANNIE / Phantom / Eva-VLA [94]–[96]	PP	Environmental commands, sensor attacks, physical variations	F/S	Physical or sensor access	Environment / sensors	Observation, trajectory, robustness state	D, Sim, Robot	Embodied VLM/VLA-adjacent systems

A. Training-Time, Supply-Chain, and Backdoor Attacks

Training-time attacks arise because VLA systems are adapted from large robot datasets, demonstrations, and fine-

tuning corpora. A poisoned demonstration set may look normal during training while associating a trigger with a gripper primitive or a small delta-pose drift. At deployment, the robot may still perform clean tasks, yet the trigger causes a release, freeze, wrong-object action, or gradual trajectory shift.

VLA backdoor work has therefore moved beyond generic task disruption. The common mechanism is trigger-conditioned control of a robot-relevant representation while clean behavior remains plausible. Some studies target triggered action deviations or physical-object-conditioned goals [38], [39]. Others bind triggers to reusable primitives, long-horizon sequences, static action states, or chunk-level drift [37], [40]–[42]. Recent work further broadens the attack object: INFUSE implants persistent backdoors in fine-tune-insensitive modules, State Backdoor uses the robot arm’s initial state as the trigger, and FlowHijack targets vector-field dynamics in flow-matching VLAs [83]–[85]. The target is no longer a label; it is the command object or generative process the robot will execute.

The robotic issue is temporal control. A backdoor that fires a primitive at the wrong decision window differs from one that slowly biases a chunk, and both differ from a trigger that changes a high-level goal, initial robot state, persistent module, or continuous action field. Direct VLA studies are stronger than classifier analogies because they measure action behavior and often clean-task retention, but the settings remain narrow: most results use curated manipulation suites, controlled triggers, and limited embodiments. Current evidence still needs broader tests across data filtering, downstream fine-tuning, tokenized versus continuous action heads, chunked policies, flow/diffusion policies, and real controller clipping. Classic poisoning/backdoor work [97]–[100] and RL backdoors [101]–[104] explain the supply-chain mechanism; direct VLA work is what establishes the robotics claim.

B. Perception, Multimodal, and Physical Attacks

Visual and physical attacks are the most natural bridge from adversarial ML to VLA robotics, but fooling an image model is not enough. The robotics claim begins when a perturbation survives the perception-to-action chain: a patch or object texture changes which object the robot grounds, shifts the action trajectory, and is then either corrected by feedback or passed to the controller as wrong-object motion [105]–[108].

Early VLA patch work made this shift explicit by optimizing action discrepancy, position-aware objectives, and targeted manipulation behavior rather than image classification error [34]. Other work attacks earlier representations: VLA-Fool studies textual, visual, and cross-modal misalignment, while EDPA and ADVLA target visual embeddings, projected features, attention, and vision–language alignment [35], [43], [87]. Transfer-oriented work asks whether shared feature-space, attention, semantic, or visual-proprioception structure can carry attacks beyond one model-specific action head [44], [88]. Physical realism appears in object-bound texture attacks and in partially observable settings where the adversary sees only a short trajectory prefix [45], [89].

The mechanism sequence is now clearer. Action-space patches ask whether visual perturbations can change robot motion. Embedding and attention attacks ask whether the shared

multimodal representation is enough when the action head is unknown. Transferable patches ask whether the attack survives changes in architecture, fine-tuning, viewpoint, and sim-to-real setting. 3D texture attacks ask whether the manipulated object can carry the perturbation. Partial-observation attacks ask whether the attack can be planned without knowing the full future rollout.

The unresolved issue is not whether vision can be perturbed. Classic adversarial-example work established the digital mechanism [57], [109]–[111]. Patch and robust physical-example work then showed why physical realization changes the threat model [58]–[60], [112]. Universal and robust-optimization work explains transfer and defense assumptions in this background literature [113], [114]. Detector and wearable-object attacks extend the concern beyond image classification [115]–[118]. 3D, LiDAR, and multi-sensor settings add geometry and sensor-fusion constraints [119]–[123]. Audio-command attacks show a parallel source-confusion path for spoken interfaces [124]–[127]. What remains uncertain is how reliably a physical perturbation changes grounding, action, trajectory, and recovery across robot embodiments and controllers.

C. Instruction, Planning, and Reasoning Attacks

Instruction attacks matter because VLA systems treat language as a control input, not only as a conversation channel. In scene-text hijacking, the user asks for a normal task, but text in the environment or a small edit in the instruction changes object binding, subgoal order, or trajectory preference. A subtler case is command-preserving redirection: the instruction remains close to the intended task while the physical path moves toward a different outcome.

RoboGCG adapts adversarial prompt optimization to VLA control authority and shows that a prompt-level intervention at rollout start can influence action reachability over time [33]. VLA-Fool includes textual and cross-modal misalignment as well as visual perturbations [35]. SABER makes the instruction threat model more practical by using a black-box edit process with bounded character, token, or prompt changes and rollout-level feedback across multiple VLA models [36]. Q-DIG approaches red-teaming differently, using quality-diversity search to generate natural, task-relevant instructions that expose robot-policy failures and can support robustness improvement [86]. Trajectory-level redirection studies show why instruction similarity is not enough: a prompt can remain close to the intended command while redirecting the physical rollout [47]. Prompt and multimodal red-teaming work supplies useful background on optimization, jailbreak transfer, and benchmark design, but robot-level claims require VLA rollouts [128]–[132] [133]–[137].

Reasoning-enabled VLAs add another target between language and action. TRAP shows that CoT or intermediate reasoning traces can be hijacked by physical patches to steer downstream actions without changing the user’s instruction [46]. Altered-thought studies indicate that entity references in VLA reasoning traces can causally affect robot manipulation success, while non-reasoning control models may be immune to that channel [138]. CoT-VLA illustrates the constructive

side of this interface by using visual chain-of-thought reasoning before action generation [139]. ReasonBreak extends the concern to autonomous-driving-style VLA systems, evaluating black-box textual corruptions against reasoning and trajectory safety metrics in closed-loop simulation [140]. In these architectures, reasoning text is not merely explanation; it can become state that the controller ultimately acts on.

Embodied jailbreak and planning benchmarks provide useful context. BadRobot, RoboPAIR, and RoboJailBench evaluate embodied jailbreaks where success is measured closer to action than text-only refusal [90], [91], [93]. CHAI, SafeAgentBench, and EARBench add physical command hijacking and risk-aware planning settings [92], [141], [142]. Multimodal jailbreak benchmarks further clarify source and modality confusion, but remain contextual unless the outcome reaches robot action [143]–[146]. Their limitation for this survey is target specificity: when the victim is an LLM-controlled planner rather than a VLA policy, the result motivates the attack surface but does not prove VLA action vulnerability. Direct VLA studies still need to test how instruction and reasoning attacks behave under prompt hierarchy, source labels, physical context changes, and closed-loop recovery.

D. Action-Level and Closed-Loop Attacks

Action representation is the robotics core of the survey. In an action-freezing attack, the perception and language inputs look plausible, but the policy repeatedly emits a static or blocked action state, causing the robot to stall or fail to release control at the right moment. Chunk drift is subtler: each predicted delta-pose is only slightly biased, but the robot executes a multi-step chunk before new observations can correct the drift.

Autoregressive action-token policies expose token probabilities and discretized gripper or pose bins. A prompt or patch can change which token is selected, and early token errors can propagate through subsequent generation. RT-1, RT-2, and OpenVLA illustrate tokenized robot-action interfaces whose vocabularies and detokenizers become attack-relevant objects [2], [3], [6]. SpatialVLA and UniVLA show how spatial grids or unified token streams change the token-to-control object [13], [67]. FAST, BeT, and VQ-BeT make action tokenization itself an explicit design variable [23], [77], [78]. Parallel action chunks create a different risk: the model commits to several future actions before new observations can correct the error. ACT and OpenVLA-OFT show why chunk length, temporal ensemble, and parallel decoding are security variables as well as efficiency variables [22], [68]. RDT-1B and FAST-style VLA training further connect chunked action prediction to high-frequency or diffusion-based robot control [23], [24]. SilentDrift directly exploits this intra-chunk open-loop property [42].

Waypoint and trajectory policies expose geometric objectives such as end-effector displacement, curvature, unsafe workspace entry, or targeted path redirection [34], [47]. Planner-mediated systems make object binding, generated code, value maps, and waypoint plans inspectable. CLIPort and Transporter Networks show how spatial pick-place or transport targets become policy objects [20], [69]. PerAct,

Act3D, and RVT expose 3D action maps or gripper-pose predictions as target representations [70]–[72]. Code as Policies, ProgPrompt, and VoxPoser make generated code, subgoals, and value maps inspectable planner outputs [17]–[19]. Diffusion and flow policies shift the target from a discrete token to a conditional trajectory distribution, denoising path, velocity field, or sampling process. Diffusion Policy and DP3 model action sequences with conditional diffusion [21], [73]. π_0 and 3D Diffuser Actor emphasize flow or 3D denoising structure [8], [74]. RDT-1B, MDT, and H3DP extend this family with diffusion transformers or hierarchical diffusion policies [24], [75], [76]. Generalist policy work defines cross-embodiment assumptions [5], [7], [9], [147]. Humanoid-oriented foundation models add another embodiment axis [14]. Robot-data collections expose provenance and embodiment choices that shape transfer claims [148]–[150] [151], [152].

Whether an action-level attack becomes physically consequential depends on controller and recovery design. A temporal ensemble can damp high-frequency perturbations but may also prolong a poisoned chunk. A high re-planning frequency can recover from one-frame perception attacks yet remain vulnerable to repeated low-amplitude nudges. Controller latency and stale observations can turn a semantically correct command into an unsafe physical action. Safety envelopes and low-level controllers can prevent direct collision while still allowing wrong-object manipulation, unauthorized skill use, privacy leakage, or near-boundary behavior. The difficult case is not only to attack an action representation, but to show that the attack crosses the controller boundary and survives recovery.

E. System, Memory, Tool, HRI, and Persistent Attacks

VLA deployments will not be isolated policies. They will use tools, maps, object memories, user preferences, middleware, logs, and human instructions. Persistent compromise can arise when a corrupted object memory or user preference is written during one session and retrieved in a later session, causing the robot to choose the wrong object or tool even though the new instruction is benign. Another case is an untrusted tool response that changes a skill argument or authorization boundary.

Tool-agent attacks and indirect prompt injection show how untrusted context can override source boundaries and cause tool misuse or data leakage [54], [56], [153]. Related benchmarks and attack studies sharpen the same source-confusion problem for software agents [55], [154]. In robotics, the analogous failure can become an unauthorized skill call, unsafe tool argument, or middleware command. Toolformer, ReAct, Gorilla, and ToolLLM are useful as software-agent substrates [155]–[158]. WebShop and WebArena illustrate interactive task settings where tool use and environment feedback are part of the evaluation loop [159], [160]. VLA claims still require robot skills or VLA-controlled actions in the loop. ROS/ROS2 work shows that middleware, topics, services, node identity, and controller interfaces can be compromised [161]–[163]. Broader robot cybersecurity surveys frame the same problem at the deployed-system level [52], [53]. Industrial, surgical,

TABLE IV

ATTACK FAMILY BY TARGET-REPRESENTATION EVIDENCE MAP. SYMBOLS ARE QUALITATIVE EVIDENCE STRENGTH FOR DIRECT VLA ATTACK LITERATURE: 0 = NO DIRECT EVIDENCE IN THE REVIEWED SET; + = EARLY OR NARROW EVIDENCE; ++ = REPEATED EVIDENCE IN SIMULATION OR DIGITAL ROLLOUTS; +++ = STRONGER EVIDENCE WITH MULTIPLE TARGETS, TRANSFER, OR PHYSICAL/ROBOT VALIDATION. THE TABLE SUMMARIZES MECHANISM COVERAGE RATHER THAN POOLED ASR.

Attack family	Grounding / plan	Reasoning trace	Action token / chunk	Waypoint / trajectory	Diffusion / flow	Skill / tool call	Memory / privacy	Val.
Instruction/control	+++	+	++	++	0	+	0	D/Sim; limited Robot
Visual/physical	+++	+	++	+++	+	0	0	D/Sim/Robot
Backdoor/action	+	0	+++	++	+	+	+	Sim; limited Robot
Reasoning attacks	+	+++	+	++	0	0	0	D/Sim/Robot
Privacy attacks	0	0	+	++	0	0	+++	D/Sim
Embodied-adjacent	++	++	+	+	0	++	+	D/Sim/Robot

and service-robot studies show analogous weaknesses in deployed platforms [164]–[166]. HRI work shows that robots can affect human trust, authority, and physical access decisions [167]–[170].

Memory and privacy broaden the consequence space. RAG and memory poisoning show how retrieved context can be corrupted and reused later [171]–[174]. For VLA robots, multi-session embodied memory compromise remains under-developed: poisoned maps, object memories, tool logs, or preferences may survive correction and later redirect behavior. VLA membership-inference work exposes a different risk. Action likelihoods, action errors, temporal motion patterns, attention discrepancies, and trajectory structure can leak training membership [48], [49]. This is a direct VLA security problem even when no collision or task failure occurs; evaluation must connect confidentiality, persistence, authorization, and physical task performance. Having mapped the mechanisms, we next ask how strong the evidence is for each mechanism and which claims remain only adjacent or contextual.

IV. WHAT CURRENT EVIDENCE SHOWS

After identifying the mechanisms, the next question is evidentiary: which attacks have direct VLA evidence, which remain embodied-adjacent, and how far does each validation tier carry the typed attack claim? The direct VLA attack literature now supports mechanism-level conclusions, but not deployment-wide risk estimates. A useful comparison is therefore not a single leaderboard. It asks how the attack enters, which internal object changes, what robot behavior follows, and how far the validation setting carries the claim.

A. Mechanism-Level Patterns

Current work shows that VLA attacks increasingly target control-relevant internal objects, not only inputs. Visual patches can move embeddings, object grounding, visual proprioception, and trajectory objectives. Instruction edits can steer plans, shift action tokens, or redirect a trajectory while preserving the apparent task. Backdoors can bind triggers to primitives, action freezing, robot state, persistent modules, chunk-level drift, or flow-vector-field dynamics. Privacy attacks use action likelihoods and temporal traces to infer training membership. A survey organized only by input modality misses these robotics-specific pathways.

A second pattern is that the strongest results demonstrate vulnerable VLA components rather than deployed hazard rates. Many papers show that an attack changes a rollout; fewer show whether the command is clipped, delayed, rejected by a safety filter, stopped by an emergency monitor, or corrected after a new observation. A patch that changes an action head in simulation, a prompt that changes a LIBERO rollout, and a backdoor that changes a gripper primitive all matter, but they do not imply the same physical risk.

The clearest limitation is the gap between attack realism and robotics-aware evaluation. Attack papers now include black-box instruction edits, transferable patches, object-bound 3D textures, partial-observation settings, and supply-chain triggers. Evaluation still lags on controller traces, HIL studies, recovery, human-proximate settings, and adaptive attacks against deployed guardrails. Stronger attacks without controller and recovery evidence do not answer deployment questions; evaluation without adaptive attacks can understate residual risk.

B. Representative Work by Mechanism

The current direct literature separates four mechanism families that are often conflated. Instruction-control attacks enter through language but are judged by robot behavior. RoboGCG studies whether adversarial text can create low-level control authority [33]; SABER uses bounded black-box edits to induce task failure, action inflation, or constraint violations [36]; Q-DIG uses natural prompt diversity to expose policy failures [86]; and trajectory redirection tests whether a command-preserving prompt can bend the rollout toward an attacker-specified outcome [47]. These attacks share an entry surface but differ in objective, attacker cost, and degree of targeting.

Visual attacks differ by where they act in the perception-to-action chain. Action-space attacks optimize VLA-specific action discrepancy and trajectory objectives [34]. Feature-level attacks move earlier, targeting visual embeddings, projected features, attention, or vision–language alignment [43], [87]. Transfer-oriented work exploits shared feature-space, semantic, attention, or visual-proprioception structure rather than a model-specific action head [44], [88]. Object-bound textures and partially observable patches add physical constraints: the perturbation must move with the object or be optimized from a short trajectory prefix [45], [89].

Action/backdoor attacks are not merely training-time versions of patches. They target the object that the policy will

later execute: an action deviation, physical-object-triggered goal, reusable primitive, long-horizon sequence, static action state, initial robot state, persistent module, chunked delta-pose drift, or continuous action field [38]–[40], [83], [84]. Other work targets action freezing, long-horizon triggered behavior, stealthy chunk drift, and flow-matching vector-field dynamics [37], [41], [42], [85]. For comparison, the attacked action representation matters more than the trigger type alone.

Reasoning and privacy attacks expand the objective set. TRAP and altered-thought evaluations show that reasoning traces, entity references, and CoT-to-action pathways can become control-relevant attack targets [46], [138]. Reason-Break identifies a related reasoning–trajectory vulnerability in autonomous-driving VLA systems, where collision and safety metrics matter more than tabletop task success [140]. Membership inference and VLALeaks show that action likelihoods, action errors, temporal motion patterns, attention discrepancies, and trajectory structure can leak training membership [48], [49]. These attacks require confidentiality metrics rather than task-failure ASR.

C. Comparison Limits

ASR is not directly comparable across the current VLA attack literature. In one paper, ASR may mean task failure under a visual patch; in another, targeted gripper release within a reaction window; in another, action inflation; in another, trajectory redirection; and in privacy work, AUC or TPR@FPR for membership inference. The denominator also varies: image frames, task episodes, triggered episodes, candidate trajectories, model queries, or physical trials. Validation ranges from digital input-only to simulation rollout to real-robot actuation. Pooling these numbers would create false precision.

Comparable claims are therefore qualitative and typed. Direct VLA evidence supports the claim that VLA policies can be attacked through vision, language, action representations, backdoors, reasoning traces, and privacy channels. It does not yet support a universal claim that any particular attack transfers across all VLA architectures, embodiments, controllers, or safety envelopes. Embodied-adjacent evidence supports risk hypotheses about source confusion, tool misuse, middleware compromise, sensor corruption, and HRI authority ambiguity. Contextual evidence supports mechanisms and evaluation requirements, not VLA vulnerability claims by itself.

D. Strengths and Limits

The strongest direct evidence is concentrated in tabletop manipulation with a small set of open VLA targets and benchmarks. This creates a common baseline, but it weakens claims about general VLA security. Flow/diffusion policies, bimanual systems, mobile manipulation, navigation, tactile policies, and high-level tool/skill interfaces are less consistently covered. Action representations are also unevenly studied: tokenized action heads and patch-induced trajectory deviation are common, whereas diffusion/flow denoising paths, velocity fields, controller latency, temporal ensembling, recovery policies, and safety-envelope interactions are only beginning to receive direct attack evidence.

Physical validation is improving but remains narrow. Some direct VLA papers include physical robot demonstrations, but few provide enough detail on sensor stack, control frequency, calibration, lighting, object mass, latency, recovery behavior, and safety filters to make deployment claims portable. Hardware-in-the-loop remains underdeveloped as a middle tier between simulation and full physical trials.

Privacy is an emerging direct VLA attack area rather than a side issue. Membership inference papers show that VLA training data can leak through action outputs, likelihoods, attention patterns, and temporal motion signatures [48], [49]. These results support confidentiality conclusions, not physical-safety conclusions. Future work should include privacy objectives rather than treating all VLA attacks as task-failure attacks.

E. Target Model and Benchmark Concentration

Target-model concentration is informative. OpenVLA and OpenVLA-OFT appear frequently because they are open, strong, and easy to evaluate on LIBERO [6], [68], [175]. π_0 and π_0 -fast matter because flow or diffusion-style action generation changes both inference speed and attack target representation [8], [23]. SpatialVLA matters because spatial action grids and 3D spatial encoding create a different target representation from plain tokenized actions [67]. UniVLA, GR-2, GR00T N1, Magma-style multimodal agents, reasoning VLAs, Alpamayo-style driving VLAs, and VLA-Adapter broaden the evidence base, but they have not yet been tested as repeatedly as OpenVLA-style manipulation systems [12]–[15], [176] [42], [46], [88], [140].

The benchmark distribution is similarly concentrated. LIBERO is the dominant manipulation benchmark in the direct attack matrix, with RLBench, CALVIN, ManiSkill, RoboCasa, DROID, Language-Table, and Bridge-derived data serving more often as capability or transfer context [148], [177]–[180] [62], [149], [151], [152]. This concentration is useful for comparability but can overrepresent tabletop manipulation, fixed cameras, short horizons, and simplified safety envelopes. Benchmark audits and robot-data papers therefore remain relevant to attack evidence even when they are not attacks themselves [5], [150], [181], [182].

F. Evaluation Weaknesses

Clean utility is usually clearer for backdoors than for inference-time attacks, because a backdoor must remain hidden under benign inputs. Transfer is most visible in patch and benchmark papers, but transfer axes differ: model transfer, task transfer, viewpoint transfer, suite transfer, and sim-to-real transfer are not interchangeable. Query budget appears in black-box instruction work but is absent from many visual and backdoor studies. Adaptive settings remain rare: most papers test a fixed attack against an undefended or lightly defended policy, not against a deployed prompt hierarchy, source-authentication layer, action shield, or runtime monitor. Recovery is the weakest dimension. Even when physical trials are included, many studies stop at task failure or target action induction rather than asking whether the robot could stop, replan, ask for help, or recover under controller constraints.

TABLE V

CHECKLIST FOR REPORTING VLA ATTACK CLAIMS. D = DIGITAL INPUT; SIM = SIMULATION ROLLOUT; HIL = HARDWARE-IN-THE-LOOP; ROBOT = REAL ROBOT ACTUATION; F = TASK FAILURE; T = TARGETED BEHAVIOR; S = SAFETY VIOLATION; P = PRIVACY LEAKAGE; A = AUTHORIZATION MISUSE.

Item	Must report	Why it matters
Threat model	Knowledge; access; privilege; timing; query/physical budget	Makes ASR comparable across white-box, black-box, physical, and supply-chain settings.
Evidence stratum	Direct VLA; embodied-adjacent; contextual	Prevents adjacent sensor, LLM-agent, or ROS results from being overclaimed as VLA attacks.
Target component	Perception; grounding; planner; action head; controller; memory; tool/API	Locates which stage or interface the attack actually changes.
Target representation	Embedding; plan; reasoning trace; token; chunk; waypoint; trajectory; skill call; memory entry	Links attack mechanism to measurable VLA objects.
Objective type	F; T; S; P; A; persistence; recovery failure	Separates task degradation from security-relevant consequences.
Denominator	Frame; prompt; episode; triggered episode; trajectory; query; physical trial; membership candidate	Defines what ASR/AUC/TPR values actually count.
Validation tier	D; real sensor; Sim; HIL; Robot; human-proximate	Bounds whether the claim supports model, rollout, or deployment conclusions.
Transfer axes	Prompt; model; benchmark; action representation; embodiment; sensor; controller	Distinguishes local evidence from broader VLA risk.
Stealth and cost	Visibility; plausibility; duration; interventions; compute; queries	Connects attack success to deployability and detectability.
Recovery behavior	Replan; safe stop; user correction; memory reset; session boundary	Captures closed-loop and multi-session risk.
Controller trace	Chunk horizon; control frequency; safety filter; pre/post-controller action	Shows whether the model-level deviation crosses the controller boundary.
Severity	Safe stop; wrong object; unsafe motion; privacy/authorization breach; human proximity	Prevents all failed episodes from collapsing into one ASR.

G. Open and Early Topics

Some topics are genuinely open: multi-session memory poisoning of VLA robots, adaptive attacks against deployed safety envelopes, HRI attacks with ethically controlled human subjects, and standardized HIL benchmarks. Other topics have preliminary direct evidence but insufficient breadth: transferable visual patches, black-box instruction attacks, action-level backdoors, reasoning-chain attacks, and VLA privacy attacks. These should be described as early but real VLA evidence, not as unexplored. The comparison also shows why evaluation cannot stop at ASR: the next section turns evidence strength into reporting requirements for typed, controller-aware VLA attack claims.

V. JUDGING VLA ATTACK CLAIMS

With evidence strength separated from mechanism labels, we can ask what makes a VLA attack claim well founded. Four questions matter. What representation changed? What robot or system consequence followed? What validation setting supports the claim? Does the result survive benign utility, transfer, controller interaction, and recovery? Table V summarizes the information needed to answer these questions, but the goal is interpretation rather than paperwork.

A. Typed Success Rather Than One ASR

Untyped ASR is insufficient because VLA attacks have different goals. A patch-induced task failure, a targeted gripper release, an action freeze, a redirected trajectory, an unauthorized skill call, and membership inference are not the same event. Typed rates can be derived from the success definition in Section II-E. Let $\kappa_O(\tau^\delta)$ indicate that adversarial rollout τ^δ satisfies objective type O , let $\sigma_C(\tau^\delta)$ indicate violation of consequence class C , and let $\rho_H(\tau^\delta)$ indicate that the

attack effect persists after horizon H feedback steps, replans, memory updates, or sessions. For N trials,

$$\begin{aligned}
 \text{ASR}_O &= \frac{1}{N} \sum_{i=1}^N \mathbf{1}\{\kappa_O(\tau_i^\delta) = 1\}, \\
 \text{SVR}_C &= \frac{1}{N} \sum_{i=1}^N \mathbf{1}\{\sigma_C(\tau_i^\delta) = 1\}, \\
 \text{PR}_H &= \frac{1}{N} \sum_{i=1}^N \mathbf{1}\{\rho_H(\tau_i^\delta) = 1\}.
 \end{aligned} \tag{4}$$

Here ASR_O may refer to untargeted failure, targeted object choice, gripper primitive induction, action freezing, trajectory redirection, privacy membership inference, unauthorized skill invocation, or persistent memory compromise. SVR_C measures consequence-specific violations such as unsafe action, collision/contact risk, privacy leakage, or authorization failure. PR_H is required for backdoors, memory attacks, repeated instruction edits, action-chunk drift, and attacks that exploit recovery behavior.

For robot deployments, success should also be conditioned on the controller and recovery system. Let $r(\tau^\delta)$ indicate that a recovery opportunity occurred, $q(\tau^\delta)$ that the controller or safety filter rejected the command, d_{pre} and d_{post} be pre- and post-controller deviations from the benign action or trajectory, and s_{min} be the minimum signed safety margin along the rollout. Useful controller-aware metrics include

$$\begin{aligned}
 \text{ASR}_{O|r} &= \Pr(\kappa_O(\tau^\delta) = 1 \mid r(\tau^\delta) = 1), \\
 \text{CRR} &= \Pr(q(\tau^\delta) = 1), \\
 \Delta_{\text{ctrl}} &= d_{\text{post}} - d_{\text{pre}}.
 \end{aligned} \tag{5}$$

Here $\text{ASR}_{O|r}$ is recovery-conditioned ASR, CRR is controller rejection rate, and Δ_{ctrl} measures whether the controller attenuates or amplifies the model-level deviation; the probabilities are estimated over the evaluated rollouts. Time-to-violation and s_{min} are especially important for human-proximate robots:

two attacks with the same ASR may differ sharply if one violates the safety envelope after one control cycle while another remains far from contact.

B. Action Representation Changes the Claim

The same attack can mean different things under different action representations. For autoregressive action-token policies, the relevant question is whether the first changed token propagates through token-to-control mapping and subsequent tokens [2], [3], [6], [23], [67]. For parallel action chunks, the key question is how much open-loop exposure exists before new observations can correct the command [22], [24], [68]. For waypoint or trajectory policies, displacement, curvature, workspace boundary crossing, and recovery latency matter more than token accuracy [69]–[72]. For diffusion or flow policies, the claim depends on whether the target is the conditional representation, denoising path, velocity field, sampler, trajectory distribution, or final controller command [8], [21], [73]–[75], [85].

The propagation mechanism differs by representation. In autoregressive action-token policies, an early token error can be amplified by the detokenizer and by subsequent conditional token generation; the relevant variables are action frequency, token bin width, and whether gripper and pose tokens are coupled [23], [77], [78]. In parallel action-chunk policies, the attack may corrupt a multi-step command before new observations arrive; chunk horizon, temporal ensemble, and replan rate determine the open-loop exposure [22], [42], [68]. In waypoint and trajectory policies, the attack is geometric: displacement, curvature, workspace boundary crossing, and tracking-controller error are more informative than token accuracy. In diffusion or flow policies, the target may be the conditional representation, denoising path, velocity field, sampler, or final trajectory distribution; small distributional changes can be smoothed by sampling but can also create plausible near-boundary trajectories. Flow-matching backdoors add a further check: whether the malicious vector field remains kinematically similar to benign dynamics while still reaching the targeted behavior [85]. In skill/API policies, the safety problem is often authorization and precondition bypass: selecting a valid but unauthorized skill can be more consequential than perturbing a low-level motor command.

The physical controller must be part of the claim. A high-level action may be rejected, clipped, smoothed, delayed, or transformed by a low-level controller, safety filter, control barrier function, recovery policy, or emergency stop [79]–[82]. If only simulator actions are shown, claims about collision/contact risk or human safety should remain tentative. Pre- and post-controller traces reveal whether the attack is attenuated, amplified, or converted into a different consequence.

C. Claim Checks by Attack Family

Different attack families need different claim checks. For *instruction-control attacks*, the key issue is whether a language change actually causes action divergence under a stated query or edit budget. RoboGCG-style control-authority claims

require a different denominator from SABER-style task success reduction, Q-DIG-style failure discovery, or trajectory-redirection claims [33], [36], [47], [86]. A useful result ties together the instruction pair, target policy, rollout seed, attack budget, action divergence, consequence, and recovery outcome.

For *visual and physical attacks*, the key issue is where the perturbation lives and what representation it changes: digital noise, localized patch, printed patch, object texture, scene object, or sensor-path interference; action loss, embedding loss, attention loss, semantic misalignment, or trajectory loss. This distinction separates action-space and feature-level VLA patch attacks [34], [43], [87]. It also separates transferable patch attacks, object-surface attacks, and partially observable patch attacks [44], [45], [88], [89].

For *backdoor and supply-chain attacks*, the key issue is whether the trigger survives realistic adaptation while preserving clean utility. Triggered metrics should separate failure, goal completion, primitive induction, and physical-object-conditioned behavior [38]–[40]. They should also distinguish long-horizon triggered behavior, action freezing, chunk drift, initial-state triggers, persistent base-model compromise, and vector-field hijacking [37], [41], [42], [83]–[85]. A backdoor with poor benign performance is easier to detect and less informative about realistic supply-chain risk.

For *reasoning and privacy attacks*, the metric is not ordinary task ASR. Reasoning attacks should name the attacked reasoning object, such as natural-language CoT, object entity references, bounding-box traces, subgoal plans, or driving rationales, and then measure whether the changed reasoning causally changes action [46], [138], [140]. Privacy attacks should report the membership unit, access regime, candidate distribution, and confidence metric, because sample-level and trajectory-level membership inference are not comparable to physical task failure [48], [49].

D. Representation Context

VLA attacks inherit representation assumptions from VLMs, action policies, and adversarial ML. Contrastive and few-shot vision-language backbones explain why embeddings, attention, and cross-modal alignment are plausible VLA targets [183]–[185]. Instruction-tuned VLMs provide additional context for multimodal alignment and prompt-conditioned representations [186]–[188]. Robot-policy substrates clarify why the output side is equally representation-dependent: imitation policies, tokenized behavior models, 3D action maps, diffusion policies, and generalist robot policies make different objects measurable [21], [69], [77], [78], [189] [7]–[9], [24]. Query-efficient and transfer-oriented adversarial methods help design black-box evaluation [190]–[194] [195]–[198]. These works explain mechanisms; a VLA claim still requires a VLA rollout, action representation, or VLA privacy outcome.

E. Physical Validation

Physical validation determines how far a claim can travel. Digital input-only tests are useful for isolating mechanisms such as attention, likelihood, embeddings, prompt transfer,

TABLE VI

EVALUATION RESOURCES AND THE CLAIMS THEY CAN SUPPORT FOR VLA ATTACK STUDIES. ● = DIRECTLY SUPPORTS; ○ = PARTIALLY SUPPORTS OR REQUIRES ADDITIONAL ASSUMPTIONS; ◦ = NOT DIRECTLY SUPPORTED. A RESOURCE SUPPORTS ONLY THE EVIDENCE LEVEL IT MEASURES; ADVERSARIAL OVERLAYS AND ROBOT ROLLOUTS ARE NEEDED BEFORE CAPABILITY OR SENSOR TESTS BECOME DIRECT VLA ATTACK EVIDENCE.

Resource category	Examples	G/P	CL act.	Ctrl.	Phys.	Adv.	Rec.	Maximum supported claim
Capability benchmarks	AI2-THOR; ALFRED; Habitat [199]–[201]	●	○	◦	◦	◦	◦	Task capability under nominal conditions
Manipulation simulators	RLBench; CALVIN; LIBERO; ManiSkill [175], [177]–[179]	●	●	○	◦	○	◦	Simulated rollout-level effects
Data / policy resources	RoboCasa; DROID; BridgeData; RoboMimic [148], [149], [151], [180]	○	○	◦	◦	◦	◦	Data diversity, provenance, transfer context
Tool-agent red-team suites	ToolEmu; AgentDojo; InjecAgent [55], [56], [153]	○	◦	◦	◦	●	◦	Source confusion and tool misuse
Safety / risk-planning suites	SafeAgentBench; EARBench [141], [142]	●	○	◦	◦	○	○	Hazard-aware planning behavior
Embodied jailbreak benchmarks	BadRobot; RoboPAIR; RoboJailBench [90], [91], [93]	●	○	◦	○	●	○	Embodied harmful-goal execution
Direct VLA attack studies / suites	RoboGCG; VLA-Fool; SABER; FreezeVLA; DropVLA; AttackVLA [33], [35]–[37], [40], [41]	●	●	○	○	●	○	Model- and platform-specific attack effects
Sensor / physical testbeds	CHAI; ANNIE; Eva-VLA; Phantom Menace [92], [94]–[96]	○	○	◦	○	●	◦	Sensor-path and physical robustness
Benchmark audits	Benchmark audit studies [181]	○	○	◦	◦	○	◦	Benchmark validity and shortcut risk

or membership features. Real-sensor input shows that a perturbation can be captured by a real camera, microphone, LiDAR, or robot sensor, but it does not necessarily show robot consequence. Simulation rollout measures closed-loop behavior at scale. Hardware-in-the-loop exposes real sensors, controller timing, middleware, or safety filters while limiting hazard. Real-robot actuation shows physical execution. Human-proximate validation is required before making claims about people inside the safety boundary. Hybrid attacks should name both the delivery channel and the consequence setting, for example digital prompt with real-robot actuation, or physical patch with simulation-only consequence.

F. Benchmark Claims

Benchmarks must be matched to claims. Table VI summarizes which evaluation resources can support grounding/planning, closed-loop action, controller interaction, physical actuation, adversarial evaluation, and recovery claims. LIBERO, CALVIN, RLBench, ManiSkill, and RoboCasa can support manipulation and rollout comparisons, but they do not by themselves establish physical safety [175], [177]–[180]. DROID-derived settings and broader imitation-learning suites add data diversity and policy-learning context [148], [151], [182]. Offline RL, multi-task, and simulation frameworks provide additional benchmark substrates rather than attack evidence by themselves [202]–[204]. AgentDojo, ToolEmu, InjecAgent, and RAG benchmarks support source-boundary and tool-risk analogies, not robot consequences. Embodied jailbreak benchmarks support action-boundary evaluation for planners and LLM-controlled robots, but direct VLA claims require VLA policies or VLA-enabled robot systems. A credible attack benchmark needs paired benign/adversarial tasks, threat assumptions, typed ASR, severity labels, transfer tests, physical validation, and recovery metrics.

G. Adaptive and Responsible Evaluation

Guardrails change the attack objective. Prompt hierarchy turns injection into source-boundary confusion; memory hy-

giene turns poisoning into persistence; tool permissions turn misuse into argument selection or authorization-boundary crossing; safety filters turn direct collision objectives into near-boundary, semantic misuse, or recovery-exhaustion objectives. Attack papers should state whether the attacker knows the guardrail, receives rejection feedback, adapts to it, or acts only through the physical environment.

Because VLA attacks may transfer to real robots, papers should publish sanitized prompts, aggregate results, threat models, and defense-relevant analysis while withholding reusable high-risk payloads or hardware exploit recipes when needed. Responsible disclosure is part of evaluation quality, not an afterthought. These evaluation requirements expose the remaining gaps: deployment-relevant VLA attack conclusions require stronger physical validation, action-representation transfer tests, privacy metrics, and adaptive studies against recovery and guardrails.

VI. RESEARCH ROADMAP AND LIMITS

The evaluation criteria above reveal a deployment gap: many attacks change model outputs or simulated rollouts, but fewer show how the effect survives controller execution, recovery, transfer, privacy constraints, and adaptive defenses in deployed robot systems.

A. Physically Grounded Attacks

Physical grounding is essential because VLA attacks ultimately claim effects on robot motion, tool use, environment state, or human proximity. Current work has moved beyond digital-only examples: several patch, texture, backdoor, and embodied red-team studies include simulation or real-robot demonstrations. Progress now depends less on simply adding physical trials than on making those trials informative. Calibrated cameras, lighting and viewpoint variation, object mass and friction, controller frequency, safety filters, time-to-violation, and recovery opportunities should be part of the evaluated claim.

B. HIL and Real-Robot Recovery

Recovery is the most underdeveloped robotics question. Simulation gives scale, and full physical deployment gives realism, but hardware-in-the-loop can expose real sensors, middleware timing, safety filters, controller latency, and emergency-stop logic without requiring unsafe full deployment. Future studies should show when an attack first changes representation, whether the controller clips or rejects it, whether the robot recovers after new observations, and whether human correction changes the outcome. A VLA attack that fails after one replan and a VLA attack that survives repeated recovery attempts are different risks.

C. Cross-Embodiment and Action-Representation Transfer

Transfer matters because VLA systems are defined by embodiment as much as by model name. Existing work has begun to test transfer across prompts, models, viewpoints, and simulation-to-real settings, while robot-learning work has long treated cross-task, cross-dataset, and cross-embodiment generalization as central design problems [5], [62], [148], [205], [206] [7], [9], [14], [151], [152]. Transfer across action representations is still thin. A prompt attack may transfer across language-conditioned token policies but fail against a flow policy; a patch may transfer across visual encoders but not camera placements; a chunk-drift backdoor may be irrelevant to a one-step replanner. Future suites should vary tokenizer, chunk length, temporal ensemble, diffusion/flow sampler, waypoint controller, robot morphology, camera geometry, and low-level safety envelope.

D. Memory, Tool, Middleware, and HRI Attacks

Deployed VLA robots will not be stateless policies. They will retrieve maps, object memories, user preferences, logs, tool outputs, and social cues. Adjacent evidence from RAG, software agents, ROS/ROS2, and HRI shows credible routes for corruption [56], [163], [167], [173], but direct VLA evidence remains thin. Future work should measure robot-level effects: unauthorized skill use, corrupted maps, persistent user preference changes, unsafe tool arguments, privacy leakage, and recovery after correction.

E. Privacy as a First-Class VLA Attack Objective

Robot actions can reveal training data, user routines, scenes, and task histories. Membership inference work shows that VLA actions and trajectories can leak training membership [48], [49]. The field now needs privacy tests beyond membership: property inference, dataset provenance, user routine leakage, private-scene memorization, and cross-session log leakage. These are direct VLA security questions even when no robot collision or task failure occurs.

F. Adaptive Attacks and Responsible Artifacts

First-generation defenses will use prompt hierarchy, source labels, retrieval filters, tool permissions, runtime monitors, action shields, control barrier functions, and human approvals.

Attack research must therefore include adaptive settings: what the attacker knows, what feedback is available, whether queries are budgeted, and whether benign utility is preserved. At the same time, VLA attack artifacts can be unusually transferable to physical systems. Benchmarks should separate public scoring code and sanitized examples from high-risk payloads, physical exploit details, or deployment-specific trigger recipes.

G. Limitations of This Review

This review fixes its evidence set at the 16 June 2026 search date, so later arXiv releases, renamed project pages, and venue updates are outside scope. Google Scholar and Semantic Scholar ranking can bias discovery toward highly visible papers, while targeted-name searches can miss attacks that use different terminology. We searched English-language sources and may miss non-English reports. Several direct VLA attacks are recent preprints or workshop papers; their claims may change after peer review, and project-page or repository information is weaker evidence than archival text.

The supplementary materials document inclusion choices and supporting notes, but they do not reproduce experiments, inspect every codebase, or verify every ASR number. Some judgments remain partly subjective, especially for target representations, transfer, and validation details. The descriptive observations should therefore be read as a map of an emerging literature, not as a meta-analysis; we do not pool ASR values across attack objectives, denominators, validation tiers, or robot platforms. The conclusion returns to the central claim: VLA attack evidence becomes robotics evidence only when a changed representation survives the robot loop and reaches a typed closed-loop consequence.

VII. CONCLUSION

VLA attacks should be judged at the level of closed-loop consequences, not isolated model outputs. An adversarial effect may enter through perception, language, memory, tools, environment state, action representations, or human interaction. It may first appear as a changed object binding, plan, reasoning trace, action token, chunk, trajectory, tool call, or memory entry. The security claim becomes a robotics claim only when that change crosses the controller boundary, interacts with feedback and recovery, and reaches a task, safety, authorization, privacy, or persistence violation.

The emerging literature already shows that direct VLA systems can be attacked through patches, black-box instructions, action freezing, action-level and long-horizon backdoors, transferable and 3D object perturbations, reasoning-chain hijacking, trajectory redirection, and membership inference. Embodied-adjacent work usefully motivates risks around planners, tools, sensors, middleware, and HRI, while contextual work explains mechanisms and evaluation design. These levels should remain separate: task failure, targeted action, safety violation, privacy leakage, unauthorized skill use, persistence, and recovery failure are different predicates and should not be collapsed into one ASR.

For VLA systems, an attack claim is ultimately a claim about what survives the robot loop. Future work should test

whether attacks transfer across action representations, remain consequential under physical or HIL recovery protocols, and require privacy-aware metrics rather than task ASR. It should also evaluate adaptive attackers against deployed guardrails while releasing artifacts in forms that support defense without enabling misuse.

REFERENCES

- [1] D. Driess, F. Xia, M. S. M. Sajjadi, C. Lynch, A. Chowdhery, B. Ichter, A. Wahid, J. Tompson, Q. Vuong, T. Yu, W. Huang, Y. Chebotar, P. Sermanet, D. Duckworth, S. Levine, V. Vanhoucke, K. Hausman, M. Toussaint, K. Greff, A. Zeng, I. Mordatch, and P. Florence, “PaLM-E: An embodied multimodal language model,” in *Proceedings of the 40th International Conference on Machine Learning*, vol. 202, pp. 8469–8488, PMLR, 2023.
- [2] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, J. Dabis, C. Finn, K. Gopalakrishnan, K. Hausman, A. Herzog, J. Hsu, *et al.*, “RT-1: Robotics transformer for real-world control at scale,” 2022.
- [3] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, X. Chen, K. Choremanski, T. Ding, D. Driess, A. Dubey, C. Finn, *et al.*, “RT-2: Vision-language-action models transfer web knowledge to robotic control,” 2023.
- [4] Y. Jiang, A. Gupta, Z. Zhang, G. Wang, Y. Dou, Y. Chen, L. Fei-Fei, A. Anandkumar, Y. Zhu, and L. Fan, “VIMA: General robot manipulation with multimodal prompts,” in *Fortieth International Conference on Machine Learning*, 2023.
- [5] A. O’Neill, A. Rehman, A. Maddukuri, A. Gupta, A. Padalkar, A. Lee, A. Pooley, A. Gupta, A. Mandlekar, A. Jain, *et al.*, “Open x-embodiment: Robotic learning datasets and RT-X models,” 2023.
- [6] M. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. Foster, G. Lam, P. Sanketi, Q. Vuong, T. Kollar, B. Burchfiel, R. Tedrake, D. Sadigh, S. Levine, P. Liang, and C. Finn, “OpenVLA: An open-source vision-language-action model,” 2024.
- [7] D. Ghosh, H. R. Walke, K. Pertsch, K. Black, O. Mees, S. Dasari, J. Hejna, T. Kreiman, C. Xu, J. Luo, Y. L. Tan, L. Y. Chen, Q. Vuong, T. Xiao, P. R. Sanketi, D. Sadigh, C. Finn, and S. Levine, “Octo: An open-source generalist robot policy,” in *Proceedings of Robotics: Science and Systems*, (Delft, Netherlands), 2024.
- [8] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter, S. Jakubczak, T. Jones, L. Ke, S. Levine, A. Li-Bell, M. Mothukuri, S. Nair, K. Pertsch, L. X. Shi, J. Tanner, Q. Vuong, A. Walling, H. Wang, and U. Zhilinsky, “pi0: A vision-language-action flow model for general robot control,” 2024.
- [9] K. Bousmalis, G. Vezzani, D. Rao, C. Devin, A. X. Lee, M. Bauza, T. Davchev, Y. Zhou, A. Gupta, A. Raju, *et al.*, “RoboCat: A self-improving generalist agent for robotic manipulation,” in *Transactions on Machine Learning Research*, 2023.
- [10] H. Wu, Y. Jing, C. Cheang, G. Chen, J. Xu, X. Li, M. Liu, H. Li, and T. Kong, “Unleashing large-scale video generative pre-training for visual robot manipulation,” in *International Conference on Learning Representations*, 2024.
- [11] X. Li, M. Liu, H. Zhang, C. Yu, J. Xu, H. Wu, C. Cheang, Y. Jing, W. Zhang, H. Liu, H. Li, and T. Kong, “Vision-language foundation models as effective robot imitators,” in *International Conference on Learning Representations*, 2024.
- [12] C.-L. Cheang, G. Chen, Y. Jing, T. Kong, H. Li, Y. Li, Y. Liu, H. Wu, J. Xu, Y. Yang, M. Zhu, and H. Zhang, “GR-2: A generative video-language-action model with web-scale knowledge for robot manipulation,” 2024.
- [13] Y. Wang, X. Li, W. Wang, J. Zhang, Y. Li, Y. Chen, X. Wang, and Z. Zhang, “Unified vision-language-action model,” 2025.
- [14] NVIDIA, J. Bjorck, F. Castañeda, N. Cherniadev, X. Da, R. Ding, L. Fan, Y. Fang, D. Fox, F. Hu, *et al.*, “GR00T N1: An open foundation model for generalist humanoid robots,” 2025.
- [15] Y. Wang, P. Ding, L. Li, C. Cui, Z. Ge, X. Tong, W. Song, H. Zhao, W. Zhao, P. Hou, S. Huang, Y. Tang, W. Wang, R. Zhang, J. Liu, and D. Wang, “VLA-Adapter: An effective paradigm for tiny-scale vision-language-action model,” 2025.
- [16] M. Ahn, A. Brohan, N. Brown, Y. Chebotar, O. Cortes, B. David, C. Finn, K. Gopalakrishnan, K. Hausman, A. Herzog, *et al.*, “Do as i can, not as i say: Grounding language in robotic affordances,” in *Conference on Robot Learning*, 2022.
- [17] J. Liang, W. Huang, F. Xia, P. Xu, K. Hausman, B. Ichter, P. Florence, and A. Zeng, “Code as policies: Language model programs for embodied control,” in *Proceedings of the IEEE International Conference on Robotics and Automation*, pp. 9493–9500, 2023.
- [18] I. Singh, V. Blukis, A. Mousavian, A. Goyal, D. Xu, J. Tremblay, D. Fox, J. Thomason, and A. Garg, “ProgPrompt: Generating situated robot task plans using large language models,” in *Proceedings of the IEEE International Conference on Robotics and Automation*, pp. 11523–11530, 2023.
- [19] W. Huang, C. Wang, R. Zhang, Y. Li, J. Wu, and L. Fei-Fei, “VoxPoser: Composable 3d value maps for robotic manipulation with language models,” in *Proceedings of the 7th Conference on Robot Learning*, 2023.
- [20] M. Shridhar, L. Manuelli, and D. Fox, “CLIPort: What and where pathways for robotic manipulation,” in *Proceedings of the 5th Conference on Robot Learning*, pp. 894–906, PMLR, 2022.
- [21] C. Chi, S. Feng, Y. Du, Z. Xu, E. Cousineau, B. Burchfiel, and S. Song, “Diffusion policy: Visuomotor policy learning via action diffusion,” in *Robotics: Science and Systems*, 2023.
- [22] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn, “Learning fine-grained bimanual manipulation with low-cost hardware,” in *Proceedings of Robotics: Science and Systems*, 2023.
- [23] K. Pertsch, K. Stachowicz, B. Ichter, D. Driess, S. Nair, Q. Vuong, O. Mees, C. Finn, and S. Levine, “FAST: Efficient action tokenization for vision-language-action models,” 2025.
- [24] S. Liu, L. Wu, B. Li, H. Tan, H. Chen, Z. Wang, K. Xu, H. Su, and J. Zhu, “RDT-1B: A diffusion foundation model for bimanual manipulation,” 2024.
- [25] X. Qi, K. Huang, A. Panda, P. Henderson, M. Wang, and P. Mittal, “Visual adversarial examples jailbreak aligned large language models,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, pp. 21527–21536, 2024.
- [26] A. Zou, Z. Wang, N. Carlini, M. Nasr, J. Z. Kolter, and M. Fredrikson, “Universal and transferable adversarial attacks on aligned language models,” 2023.
- [27] S. Huang, N. Papernot, I. Goodfellow, Y. Duan, and P. Abbeel, “Adversarial attacks on neural network policies,” 2017.
- [28] L. Pinto, J. Davidson, R. Sukthankar, and A. Gupta, “Robust adversarial reinforcement learning,” in *Proceedings of the 34th International Conference on Machine Learning*, pp. 2817–2826, 2017.
- [29] A. Gleave, M. Dennis, C. Wild, N. Kant, S. Levine, and S. Russell, “Adversarial policies: Attacking deep reinforcement learning,” in *International Conference on Learning Representations*, 2020.
- [30] Q. Li, B. Yin, W. Huang, R. Liu, B. Zou, R. Yu, J. Ye, W. Yu, and X. Wang, “Vision-language-action safety: Threats, challenges, evaluations, and mechanisms,” 2026.
- [31] P. Sermanet, A. Majumdar, A. Irpan, D. Kalashnikov, and V. Sindhwani, “Generating robot constitutions and benchmarks for semantic safety,” 2025.
- [32] B. Zhang, Y. Zhang, J. Ji, Y. Lei, J. Dai, Y. Chen, and Y. Yang, “SafeVLA: Towards safety alignment of vision-language-action model via constrained learning,” 2025.
- [33] E. K. Jones, A. Robey, A. Zou, Z. Ravichandran, G. J. Pappas, H. Hassani, M. Fredrikson, and J. Z. Kolter, “Adversarial attacks on robotic vision language action models,” 2025.
- [34] T. Wang, C. Han, J. Liang, W. Yang, D. Liu, L. X. Zhang, Q. Wang, J. Luo, and R. Tang, “Exploring the adversarial vulnerabilities of vision-language-action models in robotics,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 6948–6958, 2025.
- [35] Y. Yan, Y. Xie, Y. Zhang, L. Lyu, H. Wang, and Y. Jin, “When alignment fails: Multimodal adversarial attacks on vision-language-action models,” 2025.
- [36] X. Wu, G. Shi, Q. Wang, Z. Li, A. S. Bedi, and D. Manocha, “SABER: A stealthy agentic black-box attack framework for vision-language-action models,” 2026.
- [37] X. Wang, J. Li, Z. Weng, Y. Wang, Y. Gao, T. Pang, C. Du, Y. Teng, Y. Wang, Z. Wu, X. Ma, and Y.-G. Jiang, “FreezeVLA: Action-freezing attacks against vision-language-action models,” 2025.
- [38] X. Zhou, G. Tie, G. Zhang, H. Wang, P. Zhou, and L. Sun, “Bad-VLA: Towards backdoor attacks on vision-language-action models via objective-decoupled optimization,” 2025.
- [39] Z. Zhou, Z. Xiao, H. Xu, J. Sun, D. Wang, and J. Zhang, “Goal-oriented backdoor attack against vision-language-action models via physical objects,” 2025.

- [40] Z. Xu, J. Li, Y. Zhao, X. Zheng, X. Ma, and Y.-G. Jiang, “DropVLA: An action-level backdoor attack on vision-language-action models,” 2025.
- [41] J. Li, Y. Zhao, X. Zheng, Z. Xu, Y. Li, X. Ma, and Y.-G. Jiang, “AttackVLA: Benchmarking adversarial and backdoor attacks on vision-language-action models,” 2025.
- [42] B. Xu, Y. Shang, B. Wang, and E. Ferrara, “SilentDrift: Exploiting action chunking for stealthy backdoor attacks on vision-language-action models,” 2026.
- [43] H. Xu, Y. S. Koh, S. Huang, Z. Zhou, D. Wang, J. Sakuma, and J. Zhang, “Model-agnostic adversarial attack and defense for vision-language-action models,” 2025.
- [44] H. Lu, Y. Yu, Y. Yang, C. Yi, Q. Zhang, B. Shen, A. C. Kot, and X. Jiang, “When robots obey the patch: Universal transferable patch attacks on vision-language-action models,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2026. arXiv:2511.21192.
- [45] J. Chen, S. Huang, J. Du, S. Chen, Y. Tian, M. Wei, C. Yu, and Z. Yin, “Tex3D: Objects as attack surfaces via adversarial 3d textures for vision-language-action models,” 2026.
- [46] Z. Huang, W. Zhu, H. Qiu, X. Ji, and W. Xu, “TRAP: Hijacking VLA CoT-reasoning via adversarial patches,” 2026.
- [47] G. Puthumanaillam, V. Dongre, P. Thangeda, H. Nayyeri, D. Hakkani-Tür, and M. Ornik, “Trajectory-level redirection attacks on vision-language-action models,” 2026.
- [48] Y. Peng, M. Li, K. Xia, R. Zhang, and A. Houmansadr, “Membership inference attacks on vision-language-action models,” 2026.
- [49] X. Luan, J. Liu, X. Li, Y. Bi, R. Wu, Z. Lei, and D. Wang, “VLALeaks: Membership inference attacks against vision-language-action models,” 2026.
- [50] R. Sapkota, Y. Cao, K. I. Roumeliotis, and M. Karkee, “Vision-language-action models: Concepts, progress, applications and challenges,” 2025.
- [51] Y. Liu, W. Chen, Y. Bai, J. Luo, X. Song, K. Jiang, Z. Li, G. Zhao, J. Lin, G. Li, W. Gao, and L. Lin, “Aligning cyber space with physical world: A comprehensive survey on embodied AI,” *IEEE/ASME Transactions on Mechatronics*, vol. 30, pp. 7253–7274, 2025.
- [52] S. Neupane, S. Mitra, I. A. Fernandez, S. Saha, S. Mittal, J. Chen, N. Pillai, and S. Rahimi, “Security considerations in AI-robotics: A survey of current methods, challenges, and opportunities,” *IEEE Access*, vol. 12, pp. 22072–22097, 2024.
- [53] J.-P. A. Yaacoub, H. N. Noura, O. Salman, and A. Chehab, “Robotics cyber security: Vulnerabilities, attacks, countermeasures, and recommendations,” *International Journal of Information Security*, vol. 21, pp. 115–158, 2022.
- [54] S. Abdelnabi, K. Greshake, S. Mishra, C. Endres, T. Holz, and M. Fritz, “Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection,” in *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security*, pp. 79–90, ACM, 2023.
- [55] Y. Ruan, H. Dong, A. Wang, S. Pitis, Y. Zhou, J. Ba, Y. Dubois, C. J. Maddison, and T. Hashimoto, “Identifying the risks of LM agents with an LM-emulated sandbox,” in *The Twelfth International Conference on Learning Representations*, 2024.
- [56] E. Debenedetti, J. Zhang, M. Balunovic, L. Beurer-Kellner, M. Fischer, and F. Tramèr, “Agentdojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents,” in *Advances in Neural Information Processing Systems Datasets and Benchmarks Track*, 2024.
- [57] I. J. Goodfellow, J. Shlens, and C. Szegedy, “Explaining and harnessing adversarial examples,” in *International Conference on Learning Representations*, 2015.
- [58] T. B. Brown, D. Mané, A. Roy, M. Abadi, and J. Gilmer, “Adversarial patch,” in *NeurIPS Workshop on Machine Learning and Computer Security*, 2017.
- [59] A. Athalye, L. Engstrom, A. Ilyas, and K. Kwok, “Synthesizing robust adversarial examples,” in *Proceedings of the 35th International Conference on Machine Learning*, pp. 284–293, 2018.
- [60] K. Eykholt, I. Evtimov, E. Fernandes, B. Li, A. Rahmati, C. Xiao, A. Prakash, T. Kohno, and D. Song, “Robust physical-world attacks on deep learning visual classification,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 1625–1634, 2018.
- [61] W. Xing, M. Li, M. Li, and M. Han, “Towards robust and secure embodied AI: A survey on vulnerabilities and attacks,” 2025.
- [62] C. Lynch, M. Khansari, T. Xiao, V. Kumar, J. Tompson, S. Levine, and P. Sermanet, “Interactive language: Talking to robots in real time,” 2022.
- [63] W. Huang, F. Xia, T. Xiao, H. Chan, J. Liang, P. Florence, A. Zeng, J. Tompson, I. Mordatch, Y. Chebotar, P. Sermanet, T. Jackson, N. Brown, L. Luu, S. Levine, K. Hausman, and B. Ichter, “Inner monologue: Embodied reasoning through planning with language models,” in *Proceedings of the 6th Conference on Robot Learning*, pp. 1769–1782, 2023.
- [64] S. Belkale, T. Ding, T. Xiao, P. Sermanet, Q. Vuong, J. Tompson, Y. Chebotar, D. Dwibedi, and D. Sadigh, “RT-H: Action hierarchies using language,” in *Proceedings of Robotics: Science and Systems*, 2024.
- [65] J. Wu, R. Antonova, A. Kan, M. Lepert, A. Zeng, S. Song, J. Bohg, S. Rusinkiewicz, and T. Funkhouser, “TidyBot: Personalized robot assistance with large language models,” *Autonomous Robots*, vol. 47, no. 8, pp. 1087–1102, 2023.
- [66] A. Saxena, A. Jain, O. Sener, A. Jami, D. K. Misra, and H. S. Koppula, “RoboBrain: Large-scale knowledge engine for robots,” 2014.
- [67] D. Qu, H. Song, Q. Chen, Y. Yao, X. Ye, Y. Ding, Z. Wang, J. Gu, B. Zhao, D. Wang, *et al.*, “SpatialVLA: Exploring spatial representations for vision-language-action models,” *arXiv preprint arXiv:2501.15830*, 2025. Accepted to RSS 2025.
- [68] M. J. Kim, C. Finn, and P. Liang, “Fine-tuning vision-language-action models: Optimizing speed and success,” *arXiv preprint arXiv:2502.19645*, 2025.
- [69] A. Zeng, P. Florence, J. Tompson, S. Welker, J. Chien, M. Attarian, T. Armstrong, I. Krasin, D. Duong, V. Sindhwani, and J. Lee, “Transporter networks: Rearranging the visual world for robotic manipulation,” in *Proceedings of the 4th Conference on Robot Learning*, 2020.
- [70] M. Shridhar, L. Manuelli, and D. Fox, “Perceiver-actor: A multi-task transformer for robotic manipulation,” in *Proceedings of the 6th Conference on Robot Learning*, 2022.
- [71] T. Gervet, Z. Xian, N. Gkanatsios, and K. Fragkiadaki, “Act3D: 3d feature field transformers for multi-task robotic manipulation,” in *Proceedings of the 7th Conference on Robot Learning*, pp. 3949–3965, 2023.
- [72] A. Goyal, J. Xu, Y. Guo, V. Blukis, Y.-W. Chao, and D. Fox, “RVT: Robotic view transformer for 3d object manipulation,” in *Proceedings of the 7th Conference on Robot Learning*, pp. 694–710, 2023.
- [73] Y. Ze, G. Zhang, K. Zhang, C. Hu, M. Wang, and H. Xu, “3d diffusion policy: Generalizable visuomotor policy learning via simple 3d representations,” in *Proceedings of Robotics: Science and Systems*, 2024.
- [74] Z. Xian, N. Gkanatsios, T. Gervet, T.-W. Ke, and K. Fragkiadaki, “3d diffuser actor: Policy diffusion with 3d scene representations,” 2024.
- [75] M. Reuss, Ö. E. Yağmurlu, F. Wenzel, and R. Lioutikov, “Multimodal diffusion transformer: Learning versatile behavior from multimodal goals,” in *Proceedings of Robotics: Science and Systems*, 2024.
- [76] Y. Lu, Y. Tian, Z. Yuan, X. Wang, P. Hua, Z. Xue, and H. Xu, “H³DP: Triply-hierarchical diffusion policy for visuomotor learning,” in *International Conference on Learning Representations*, 2026.
- [77] N. M. Shafiullah, Z. Cui, A. Altanzaya, and L. Pinto, “Behavior transformers: Cloning k modes with one stone,” in *Advances in Neural Information Processing Systems*, 2022.
- [78] S. Lee, Y. Wang, H. Etukuru, H. J. Kim, N. M. M. Shafiullah, and L. Pinto, “Behavior generation with latent actions,” in *Proceedings of the 41st International Conference on Machine Learning*, pp. 26991–27008, 2024.
- [79] A. D. Ames, X. Xu, J. W. Grizzle, and P. Tabuada, “Control barrier function based quadratic programs for safety critical systems,” *IEEE Transactions on Automatic Control*, vol. 62, no. 8, pp. 3861–3876, 2017.
- [80] M. Alshiekh, R. Bloem, R. Ehlers, B. Könighofer, S. Niekum, and U. Topcu, “Safe reinforcement learning via shielding,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2018.
- [81] J. Achiam, D. Held, A. Tamar, and P. Abbeel, “Constrained policy optimization,” in *Proceedings of the 34th International Conference on Machine Learning*, pp. 22–31, 2017.
- [82] C. Dawson, S. Gao, and C. Fan, “Safe control with learned certificates: A survey of neural lyapunov, barrier, and contraction methods for robotics and control,” *IEEE Transactions on Robotics*, vol. 39, no. 3, pp. 1749–1767, 2023.
- [83] J. Zhou, Y. Wei, R. Zhen, B. Zhao, X. Xia, R. Shao, X. Su, and S. Yang, “Inject once survive later: Backdoor vision-language-action models to persist through downstream fine-tuning,” 2026.
- [84] J. Guo, W. Jiang, Y. Lin, Y. Liu, R. Zhang, G. Lu, A. Chen, X. Han, and H. Li, “State backdoor: Towards stealthy real-world poisoning attack on vision-language-action model in state space,” 2026.

- [85] X. An, T. Luo, G. Peng, Y. Wang, K. Ren, and D. Wang, "FlowHijack: A dynamics-aware backdoor attack on flow-matching vision-language-action models," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 22879–22888, 2026. arXiv:2604.09651.
- [86] S. Srikanth, F. Liang, Y.-C. Hsu, V. Bhatt, S. Zhao, H. Chen, B. Tjanaka, M. Hwang, A. Saran, D. Seita, A. Tabrez, and S. Nikolaidis, "Red-teaming vision-language-action models via quality diversity prompt generation for robust robot policies," 2026.
- [87] N. Zhang, W. Tao, X. Xiao, Q. Sun, Y. Zheng, W. Mo, P. Wang, and N. Zhang, "Attention-guided patch-wise sparse adversarial attacks on vision-language-action models," 2025.
- [88] J. Fu, K. Jiang, J. Jia, Z. Chen, X. Chen, L. Hong, S. Gao, C. Tan, D. Yang, and W. Zhang, "VLA-Hijack: A transferable patch attack against vision-language-action models via visual proprioception hijacking," 2026.
- [89] X. Wang, M. Han, T. Hao, Y. Yang, Y.-B. Zhao, and K. Tang, "Partially observable adversarial patch attacks on vision-language-action models in robotics," *IEEE Robotics and Automation Letters*, 2026. arXiv:2606.03556.
- [90] H. Zhang, C. Zhu, X. Wang, Z. Zhou, C. Yin, M. Li, L. Xue, Y. Wang, S. Hu, A. Liu, *et al.*, "Badrobot: Jailbreaking embodied LLM agents in the physical world," in *International Conference on Learning Representations*, 2025.
- [91] A. Robey, Z. Ravichandran, V. Kumar, H. Hassani, and G. J. Pappas, "Jailbreaking LLM-controlled robots," in *Proceedings of the IEEE International Conference on Robotics and Automation*, 2025. RoboPAIR; arXiv:2410.13691.
- [92] L. Burbano, D. Ortiz, Q. Sun, S. Yang, H. Tu, C. Xie, Y. Cao, and A. A. Cardenas, "CHAI: Command hijacking against embodied AI," 2025.
- [93] D. Yeke, Y. Zhou, L. Y. Lin, H. Cai, A. Bianchi, and Z. B. Celik, "Robojailbench: Benchmarking adversarial attacks and defenses in embodied robotic agents," 2026.
- [94] Y. Huang, Z. Wang, Z. Wan, Y. Tian, H. Xu, Y. Han, and Y. Gan, "Annie: Be careful of your robots," 2025.
- [95] X. Lu, J. Chen, S. Xiao, Z. Jin, Z. Chen, H. Yu, B. Qian, R. Zhou, X. Ji, and W. Xu, "Phantom menace: Exploring and enhancing the robustness of VLA models against physical sensor attacks," 2025.
- [96] H. Liu, J. Long, J. Wu, J. Hou, H. Tang, T. Jiang, W. Zhou, and W. Yao, "Eva-VLA: Evaluating vision-language-action models' robustness under real-world physical variations," 2025.
- [97] B. Biggio, B. Nelson, and P. Laskov, "Poisoning attacks against support vector machines," in *Proceedings of the 29th International Conference on Machine Learning*, 2012.
- [98] T. Gu, B. Dolan-Gavitt, and S. Garg, "Badnets: Identifying vulnerabilities in the machine learning model supply chain," in *Neural Information Processing Systems Workshop on Machine Learning and Computer Security*, 2017.
- [99] Y. Liu, S. Ma, Y. Aafer, W.-C. Lee, J. Zhai, W. Wang, and X. Zhang, "Trojaning attack on neural networks," in *Proceedings of the Network and Distributed System Security Symposium*, 2018.
- [100] N. Carlini, M. Jagielski, C. A. Choquette-Choo, D. Paleka, W. Pearce, H. Anderson, A. Terzis, K. Thomas, and F. Tramèr, "Poisoning web-scale training datasets is practical," in *IEEE Symposium on Security and Privacy*, 2024.
- [101] P. Kiourti, K. Wardega, S. Jha, and W. Li, "TrojDRL: Evaluation of backdoor attacks on deep reinforcement learning," in *Proceedings of the 57th ACM/IEEE Design Automation Conference*, pp. 1–6, 2020.
- [102] L. Wang, Z. Javed, X. Wu, W. Guo, X. Xing, and D. Song, "BACK-DOORL: Backdoor attack against competitive reinforcement learning," in *Proceedings of the Thirtieth International Joint Conference on Artificial Intelligence*, pp. 3699–3705, 2021.
- [103] J. Cui, Y. Han, Y. Ma, J. Jiao, and J. Zhang, "BadRL: Sparse targeted backdoor attack against reinforcement learning," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, pp. 11687–11694, 2024.
- [104] E. Rathbun, A. Oprea, and C. Amato, "Adversarial inception backdoor attacks against reinforcement learning," in *Proceedings of the 42nd International Conference on Machine Learning*, pp. 51273–51296, PMLR, 2025.
- [105] Y.-C. Lin, Z.-W. Hong, Y.-H. Liao, M.-L. Shih, M.-Y. Liu, and M. Sun, "Tactics of adversarial attack on deep reinforcement learning agents," in *Proceedings of the 26th International Joint Conference on Artificial Intelligence*, pp. 3756–3762, 2017.
- [106] M. Sharif, S. Bhagavatula, L. Bauer, and M. K. Reiter, "Accessorize to a crime: Real and stealthy attacks on state-of-the-art face recognition," in *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*, pp. 1528–1540, 2016.
- [107] C. Xie, J. Wang, Z. Zhang, Y. Zhou, L. Xie, and A. Yuille, "Adversarial examples for semantic segmentation and object detection," in *Proceedings of the IEEE International Conference on Computer Vision*, 2017.
- [108] T. Sato, J. Shen, N. Wang, Y. Jia, X. Lin, and Q. A. Chen, "Dirty road can attack: Security of deep learning based automated lane centering under physical-world attack," in *30th USENIX Security Symposium*, pp. 3309–3326, 2021.
- [109] C. Szegedy, W. Zaremba, I. Sutskever, J. Bruna, D. Erhan, I. J. Goodfellow, and R. Fergus, "Intriguing properties of neural networks," in *International Conference on Learning Representations*, 2014.
- [110] N. Papernot, P. McDaniel, S. Jha, M. Fredrikson, Z. B. Celik, and A. Swami, "The limitations of deep learning in adversarial settings," in *IEEE European Symposium on Security and Privacy*, pp. 372–387, 2016.
- [111] N. Carlini and D. Wagner, "Towards evaluating the robustness of neural networks," in *IEEE Symposium on Security and Privacy*, pp. 39–57, 2017.
- [112] A. Kurakin, I. J. Goodfellow, and S. Bengio, "Adversarial examples in the physical world," in *International Conference on Learning Representations Workshop*, 2017.
- [113] S.-M. Moosavi-Dezfooli, A. Fawzi, O. Fawzi, and P. Frossard, "Universal adversarial perturbations," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 1765–1773, 2017.
- [114] A. Madry, A. Makelov, L. Schmidt, D. Tsipras, and A. Vladu, "Towards deep learning models resistant to adversarial attacks," in *International Conference on Learning Representations*, 2018.
- [115] K. Eykholt, I. Evtimov, E. Fernandes, B. Li, A. Rahmati, F. Tramèr, A. Prakash, T. Kohno, and D. Song, "Physical adversarial examples for object detectors," in *12th USENIX Workshop on Offensive Technologies*, 2018.
- [116] S.-T. Chen, C. Cornelius, J. Martin, and D. H. Chau, "ShapeShifter: Robust physical adversarial attack on faster R-CNN object detector," in *Machine Learning and Knowledge Discovery in Databases*, pp. 52–68, Springer, 2018.
- [117] S. Thys, W. Van Ranst, and T. Goedemé, "Fooling automated surveillance cameras: Adversarial patches to attack person detection," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops*, 2019.
- [118] K. Xu, G. Zhang, S. Liu, Q. Fan, M. Sun, H. Chen, P.-Y. Chen, Y. Wang, and X. Lin, "Adversarial T-Shirt! evading person detectors in a physical world," in *Computer Vision – ECCV 2020*, Springer, 2020.
- [119] C. Xiao, D. Yang, B. Li, J. Deng, and M. Liu, "MeshAdv: Adversarial meshes for visual recognition," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 6898–6907, 2019.
- [120] J. Tu, M. Ren, S. Manivasagam, M. Liang, B. Yang, R. Du, F. Cheng, and R. Urtasun, "Physically realizable adversarial examples for LiDAR object detection," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 13716–13725, 2020.
- [121] J. Sun, Y. Cao, Q. A. Chen, and Z. M. Mao, "Towards robust LiDAR-based perception in autonomous driving: General black-box adversarial sensor attack and countermeasures," in *29th USENIX Security Symposium*, pp. 877–894, 2020.
- [122] Y. Cao, N. Wang, C. Xiao, D. Yang, J. Fang, R. Yang, Q. A. Chen, M. Liu, and B. Li, "Invisible for both camera and LiDAR: Security of multi-sensor fusion based perception in autonomous driving under physical-world attacks," in *IEEE Symposium on Security and Privacy*, pp. 176–194, 2021.
- [123] Y. Cao, C. Xiao, D. Yang, J. Fang, R. Yang, M. Liu, and B. Li, "Adversarial sensor attack on LiDAR-based perception in autonomous driving," in *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*, pp. 2267–2281, 2019.
- [124] N. Carlini, P. Mishra, T. Vaidya, Y. Zhang, M. Sherr, C. Shields, D. Wagner, and W. Zhou, "Hidden voice commands," in *25th USENIX Security Symposium*, pp. 513–530, 2016.
- [125] G. Zhang, C. Yan, X. Ji, T. Zhang, T. Zhang, and W. Xu, "DolphinAttack: Inaudible voice commands," in *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, pp. 103–117, 2017.
- [126] X. Yuan, Y. Chen, Y. Zhao, Y. Long, X. Liu, K. Chen, S. Zhang, H. Huang, X. Wang, and C. A. Gunter, "CommanderSong: A systematic approach for practical adversarial voice recognition," in *27th USENIX Security Symposium*, pp. 49–64, 2018.

- [127] T. Sugawara, B. Cyr, S. Rampazzi, D. Genkin, and K. Fu, “Light commands: Laser-based audio injection attacks on voice-controllable systems,” in *29th USENIX Security Symposium*, pp. 2631–2648, 2020.
- [128] F. Perez and I. Ribeiro, “Ignore previous prompt: Attack techniques for language models,” 2022.
- [129] A. Wei, N. Haghtalab, and J. Steinhardt, “Jailbroken: How does LLM safety training fail?,” in *Advances in Neural Information Processing Systems*, 2023.
- [130] P. Chao, A. Robey, E. Dobriban, H. Hassani, G. J. Pappas, and E. Wong, “Jailbreaking black box large language models in twenty queries,” 2023.
- [131] X. Liu, N. Xu, M. Chen, and C. Xiao, “AutoDAN: Generating stealthy jailbreak prompts on aligned large language models,” in *International Conference on Learning Representations*, 2024.
- [132] M. Mazeika, L. Phan, X. Yin, A. Zou, Z. Wang, N. Mu, E. Sakhaee, N. Li, S. Basart, B. Li, D. Forsyth, and D. Hendrycks, “HarmBench: A standardized evaluation framework for automated red teaming and robust refusal,” in *Proceedings of the 41st International Conference on Machine Learning*, PMLR, 2024.
- [133] X. Shen, Z. Chen, M. Backes, Y. Shen, and Y. Zhang, ““Do Anything Now”: Characterizing and evaluating in-the-wild jailbreak prompts on large language models,” in *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security*, 2024.
- [134] A. Mehrotra, M. Zampetakis, P. Kassianik, B. Nelson, H. Anderson, Y. Singer, and A. Karbasi, “Tree of attacks: Jailbreaking black-box LLMs automatically,” in *Advances in Neural Information Processing Systems*, 2024.
- [135] K. Zhu, J. Wang, J. Zhou, Z. Wang, H. Chen, Y. Wang, L. Yang, W. Ye, N. Z. Gong, Y. Zhang, and X. Xie, “PromptBench: Towards evaluating the robustness of large language models on adversarial prompts,” 2023.
- [136] X. Li, Z. Zhou, J. Zhu, J. Yao, T. Liu, and B. Han, “DeepInception: Hypnotize large language model to be jailbreaker,” 2023.
- [137] A. Paulus, A. Zharmagambetov, C. Guo, B. Amos, and Y. Tian, “AdvPrompter: Fast adaptive adversarial prompting for LLMs,” in *Proceedings of the 42nd International Conference on Machine Learning*, 2025.
- [138] T. D. Trinh, N. Akhtar, and B. Azam, “Altered thoughts, altered actions: Probing chain-of-thought vulnerabilities in VLA robotic manipulation,” 2026.
- [139] Q. Zhao, Y. Lu, M. J. Kim, Z. Fu, Z. Zhang, Y. Wu, Z. Li, Q. Ma, S. Han, C. Finn, A. Handa, T.-Y. Lin, G. Wetzstein, M.-Y. Liu, and D. Xiang, “CoT-VLA: Visual chain-of-thought reasoning for vision-language-action models,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 1702–1713, 2025.
- [140] M. Teymorianfard, J.-P. Monteuiis, J. Petit, and A. Houmansadr, “ReasonBreak: Probing vulnerabilities in reasoning-enabled vision-language-action models for autonomous driving,” 2026.
- [141] S. Yin, X. Pang, Y. Ding, M. Chen, Y. Bi, Y. Xiong, W. Huang, S. Chen, Z. Xiang, and J. Shao, “Safeagentbench: A benchmark for safe task planning of embodied LLM agents,” 2024.
- [142] Z. Zhu, B. Wu, Z. Zhang, L. Han, Q. Liu, and B. Wu, “EARBench: Towards evaluating physical risk awareness for task planning of foundation model-based embodied AI agents,” 2024.
- [143] Y. Gong, D. Ran, J. Liu, C. Wang, T. Cong, A. Wang, S. Duan, and X. Wang, “FigStep: Jailbreaking large vision-language models via typographic visual prompts,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 39, pp. 23951–23959, 2025.
- [144] Z. Niu, H. Ren, X. Gao, G. Hua, and R. Jin, “Jailbreaking attack against multimodal large language model,” 2024.
- [145] X. Liu, Y. Zhu, J. Gu, Y. Lan, C. Yang, and Y. Qiao, “MM-SafetyBench: A benchmark for safety evaluation of multimodal large language models,” in *Computer Vision – ECCV 2024*, pp. 386–403, Springer, 2024.
- [146] Y. Li, H. Guo, K. Zhou, W. X. Zhao, and J.-R. Wen, “Images are achilles’ heel of alignment: Exploiting visual vulnerabilities for jailbreaking multimodal large language models,” in *Computer Vision – ECCV 2024*, Springer, 2024.
- [147] S. Reed, K. Zolna, E. Parisotto, S. G. Colmenarejo, A. Novikov, G. Barth-Maron, M. Gimenez, Y. Sulsky, J. Kay, J. T. Springenberg, *et al.*, “A generalist agent,” in *Transactions on Machine Learning Research*, 2022.
- [148] A. Khazatsky, K. Pertsch, S. Nair, A. Balakrishna, S. Dasari, S. Karamcheti, S. Nasiriany, M. K. Srirama, L. Y. Chen, K. Ellis, *et al.*, “DROID: A large-scale in-the-wild robot manipulation dataset,” in *Proceedings of Robotics: Science and Systems*, 2024.
- [149] H. R. Walke, K. Black, T. Z. Zhao, Q. Vuong, C. Zheng, P. Hansen-Estruch, A. He, V. Myers, K. Fang, C. Finn, and S. Levine, “BridgeData V2: A dataset for robot learning at scale,” 2023.
- [150] S. Dasari, F. Ebert, S. Tian, S. Nair, B. Bucher, K. Schmeckpeper, S. Singh, S. Levine, and C. Finn, “RoboNet: Large-scale multi-robot learning,” in *Conference on Robot Learning*, 2019.
- [151] A. Mandlekar, D. Xu, J. Wong, S. Nasiriany, C. Wang, R. Kulkarni, L. Fei-Fei, S. Savarese, Y. Zhu, and R. Martín-Martín, “What matters in learning from offline human demonstrations for robot manipulation,” in *Proceedings of the 5th Conference on Robot Learning*, 2021.
- [152] A. Mandlekar, S. Nasiriany, B. Wen, I. Akinola, Y. Narang, L. Fan, Y. Zhu, and D. Fox, “MimicGen: A data generation system for scalable robot learning using human demonstrations,” in *Proceedings of the 7th Conference on Robot Learning*, 2023.
- [153] Q. Zhan, Z. Fang, Z. Li, L. Li, and D. Kang, “InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents,” in *Findings of the Association for Computational Linguistics: ACL 2024*, 2024.
- [154] J. Yi, Y. Xie, B. Zhu, K. Hines, E. Kiciman, G. Sun, X. Xie, and F. Wu, “Benchmarking and defending against indirect prompt injection attacks on large language models,” 2023.
- [155] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, and T. Scialom, “Toolformer: Language models can teach themselves to use tools,” in *Advances in Neural Information Processing Systems*, 2023.
- [156] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, “ReAct: Synergizing reasoning and acting in language models,” in *International Conference on Learning Representations*, 2023.
- [157] S. G. Patil, T. Zhang, X. Wang, and J. E. Gonzalez, “Gorilla: Large language model connected with massive APIs,” 2023.
- [158] Y. Qin, S. Liang, Y. Ye, K. Zhu, L. Yan, Y. Lu, Y. Lin, X. Cong, X. Tang, B. Qian, *et al.*, “ToolLLM: Facilitating large language models to master 16000+ real-world APIs,” in *International Conference on Learning Representations*, 2024.
- [159] S. Yao, H. Chen, J. Yang, and K. Narasimhan, “WebShop: Towards scalable real-world web interaction with grounded language agents,” in *Advances in Neural Information Processing Systems*, 2022.
- [160] S. Zhou, F. F. Xu, H. Zhu, X. Zhou, R. Lo, A. Sridhar, X. Cheng, Y. Bisk, D. Fried, U. Alon, and G. Neubig, “WebArena: A realistic web environment for building autonomous agents,” in *International Conference on Learning Representations*, 2024.
- [161] M. Quigley, K. Conley, B. Gerkey, J. Faust, T. Foote, J. Leibs, R. Wheeler, and A. Y. Ng, “ROS: An open-source robot operating system,” in *ICRA Workshop on Open Source Software*, 2009.
- [162] B. Dieber, R. White, S. Taurer, B. Breiling, G. Caiazza, H. Christensen, and A. Cortesi, “Penetration testing ROS,” in *Robot Operating System (ROS): The Complete Reference (Volume 4)*, pp. 183–225, Springer, 2020.
- [163] G. Deng, G. Xu, Y. Zhou, T. Zhang, and Y. Liu, “On the (in)security of secure ROS2,” in *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, 2022.
- [164] D. Quarta, M. Pogliani, M. Polino, F. Maggi, A. M. Zanchettin, and S. Zanero, “An experimental security analysis of an industrial robot controller,” in *IEEE Symposium on Security and Privacy*, pp. 268–286, 2017.
- [165] T. Bonaci, J. Herron, T. Yusuf, J. Yan, T. Kohno, and H. J. Chizeck, “To make a robot secure: An experimental analysis of cyber security threats against teleoperated surgical robots,” in *arXiv preprint arXiv:1504.04339*, 2015.
- [166] V. Mayoral-Vilches, U. A. Carbajo, E. Gil-Uriarte, *et al.*, “Robot cybersecurity, a review,” *International Journal of Cyber Forensics and Advanced Threat Investigations*, 2020.
- [167] S. Booth, J. Tompkin, K. Z. Gajos, J. Waldo, H. Pfister, and R. Nagpal, “Piggybacking robots: Human-robot overtrust in university dormitory security,” in *Proceedings of the 2017 ACM/IEEE International Conference on Human-Robot Interaction*, pp. 426–434, 2017.
- [168] B. Postnikoff and I. Goldberg, “Robot social engineering: Attacking human factors with non-human actors,” in *Companion of the 2018 ACM/IEEE International Conference on Human-Robot Interaction*, pp. 313–314, 2018.
- [169] W. A. Bainbridge, J. Hart, E. S. Kim, and B. Scassellati, “The benefits of interactions with physically present robots over video-displayed agents,” in *International Journal of Social Robotics*, vol. 3, pp. 41–52, 2011.
- [170] S. P. Saunderson and G. Nejat, “Persuasive robots should avoid authority: The effects of formal and real authority on persuasion in

- human-robot interaction,” *Science Robotics*, vol. 6, no. 58, p. eabd5186, 2021.
- [171] Z. Zhong, Z. Huang, A. Wettig, and D. Chen, “Poisoning retrieval corpora by injecting adversarial passages,” in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 13764–13775, 2023.
- [172] G. Deng, Y. Liu, K. Wang, Y. Li, T. Zhang, and Y. Liu, “PANDORA: Jailbreak GPTs by retrieval augmented generation poisoning,” 2024.
- [173] W. Zou, R. Geng, B. Wang, and J. Jia, “PoisonedRAG: Knowledge corruption attacks to retrieval-augmented generation of large language models,” in *Proceedings of the 34th USENIX Security Symposium*, 2025.
- [174] S. Cho, S. Jeong, J. Seo, T. Hwang, and J. C. Park, “Typos that broke the RAG’s back: Genetic attack on RAG pipeline by simulating documents in the wild via low-level perturbations,” in *Findings of the Association for Computational Linguistics: EMNLP 2024*, 2024.
- [175] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone, “LIBERO: Benchmarking knowledge transfer for lifelong robot learning,” in *Advances in Neural Information Processing Systems*, 2023.
- [176] J. Yang, R. Tan, Q. Wu, R. Zheng, B. Peng, Y. Liang, Y. Gu, M. Cai, S. Ye, J. Jang, Y. Deng, L. Liden, and J. Gao, “Magma: A foundation model for multimodal AI agents,” 2025.
- [177] S. James, Z. Ma, D. R. Arrojo, and A. J. Davison, “RLBench: The robot learning benchmark and learning environment,” *IEEE Robotics and Automation Letters*, vol. 5, no. 2, pp. 3019–3026, 2020.
- [178] O. Mees, L. Hermann, E. Rosete-Beas, and W. Burgard, “CALVIN: A benchmark for language-conditioned policy learning for long-horizon robot manipulation tasks,” *IEEE Robotics and Automation Letters*, vol. 7, no. 3, pp. 7327–7334, 2022.
- [179] T. Mu, Z. Ling, F. Xiang, D. Yang, X. Li, S. Tao, Z. Huang, Z. Jia, and H. Su, “ManiSkill: Generalizable manipulation skill benchmark with large-scale demonstrations,” in *Advances in Neural Information Processing Systems Datasets and Benchmarks Track*, 2021.
- [180] S. Nasiriany, A. Maddukuri, L. Zhang, A. Parikh, A. Lo, A. Joshi, A. Mandlekar, and Y. Zhu, “RoboCasa: Large-scale simulation of household tasks for generalist robots,” in *Proceedings of Robotics: Science and Systems*, (Delft, Netherlands), 2024.
- [181] T. Jiang, X. Tan, S. Wheeler, L. Sun, T. W. Ayalew, and M. Walter, “What are we actually benchmarking in robot manipulation?,” 2026.
- [182] S. Nasiriany, S. Nasiriany, A. Maddukuri, and Y. Zhu, “RoboCasa365: A large-scale simulation framework for training and benchmarking generalist robots,” in *International Conference on Learning Representations*, 2026.
- [183] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, *et al.*, “Learning transferable visual models from natural language supervision,” in *Proceedings of the 38th International Conference on Machine Learning*, PMLR, 2021.
- [184] J.-B. Alayrac, J. Donahue, P. Luc, A. Miech, I. Barr, Y. Hasson, K. Lenc, A. Mensch, K. Millican, M. Reynolds, *et al.*, “Flamingo: A visual language model for few-shot learning,” in *Advances in Neural Information Processing Systems*, 2022.
- [185] J. Li, D. Li, S. Savarese, and S. Hoi, “BLIP-2: Bootstrapping language-image pre-training with frozen image encoders and large language models,” in *Proceedings of the 40th International Conference on Machine Learning*, pp. 19730–19742, PMLR, 2023.
- [186] H. Liu, C. Li, Q. Wu, and Y. J. Lee, “Visual instruction tuning,” in *Advances in Neural Information Processing Systems*, 2023.
- [187] D. Zhu, J. Chen, X. Shen, X. Li, and M. Elhoseiny, “MiniGPT-4: Enhancing vision-language understanding with advanced large language models,” 2023.
- [188] W. Dai, J. Li, D. Li, A. M. H. Tiong, J. Zhao, W. Wang, B. Li, P. Fung, and S. Hoi, “InstructBLIP: Towards general-purpose vision-language models with instruction tuning,” in *Advances in Neural Information Processing Systems*, 2023.
- [189] P. Florence, C. Lynch, A. Zeng, O. Ramirez, A. Wahid, L. Downs, A. Wong, J. Lee, I. Mordatch, and J. Tompson, “Implicit behavioral cloning,” in *Proceedings of the 5th Conference on Robot Learning*, pp. 158–168, 2022.
- [190] N. Papernot, P. McDaniel, I. Goodfellow, S. Jha, Z. B. Celik, and A. Swami, “Practical black-box attacks against machine learning,” in *Proceedings of the 2017 ACM on Asia Conference on Computer and Communications Security*, pp. 506–519, 2017.
- [191] F. Tramèr, A. Kurakin, N. Papernot, I. Goodfellow, D. Boneh, and P. McDaniel, “Ensemble adversarial training: Attacks and defenses,” in *International Conference on Learning Representations*, 2018.
- [192] P.-Y. Chen, H. Zhang, Y. Sharma, J. Yi, and C.-J. Hsieh, “ZOO: Zeroth order optimization based black-box attacks to deep neural networks without training substitute models,” in *Proceedings of the 10th ACM Workshop on Artificial Intelligence and Security*, pp. 15–26, 2017.
- [193] W. Brendel, J. Rauber, and M. Bethge, “Decision-based adversarial attacks: Reliable attacks against black-box machine learning models,” in *International Conference on Learning Representations*, 2018.
- [194] A. Ilyas, L. Engstrom, A. Athalye, and J. Lin, “Black-box adversarial attacks with limited queries and information,” in *Proceedings of the 35th International Conference on Machine Learning*, pp. 2137–2146, PMLR, 2018.
- [195] Y. Li, L. Li, L. Wang, T. Zhang, and B. Gong, “NATTACK: Learning the distributions of adversarial examples for an improved black-box attack on deep neural networks,” in *Proceedings of the 36th International Conference on Machine Learning*, pp. 3866–3876, PMLR, 2019.
- [196] M. Andriushchenko, F. Croce, N. Flammarion, and M. Hein, “Square attack: A query-efficient black-box adversarial attack via random search,” in *Computer Vision – ECCV 2020*, pp. 484–501, Springer, 2020.
- [197] F. Croce and M. Hein, “Reliable evaluation of adversarial robustness with an ensemble of diverse parameter-free attacks,” in *Proceedings of the 37th International Conference on Machine Learning*, pp. 2206–2216, PMLR, 2020.
- [198] Y. Dong, F. Liao, T. Pang, H. Su, J. Zhu, X. Hu, and J. Li, “Boosting adversarial attacks with momentum,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 9185–9193, 2018.
- [199] E. Kolve, R. Mottaghi, W. Han, E. VanderBilt, L. Weihs, A. Herrasti, D. Gordon, Y. Zhu, A. Gupta, and A. Farhadi, “AI2-THOR: An interactive 3d environment for visual AI,” 2017.
- [200] M. Shridhar, J. Thomason, D. Gordon, Y. Bisk, W. Han, R. Mottaghi, L. Zettlemoyer, and D. Fox, “ALFRED: A benchmark for interpreting grounded instructions for everyday tasks,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 10740–10749, 2020.
- [201] M. Savva, A. Kadian, O. Maksymets, Y. Zhao, E. Wijmans, B. Jain, J. Straub, J. Liu, V. Koltun, J. Malik, *et al.*, “Habitat: A platform for embodied AI research,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 9339–9347, 2019.
- [202] J. Fu, A. Kumar, O. Nachum, G. Tucker, and S. Levine, “D4RL: Datasets for deep data-driven reinforcement learning,” in *Advances in Neural Information Processing Systems Datasets and Benchmarks Workshop*, 2020.
- [203] T. Yu, D. Quillen, Z. He, R. Julian, K. Hausman, C. Finn, and S. Levine, “Meta-World: A benchmark and evaluation for multi-task and meta reinforcement learning,” in *Conference on Robot Learning*, 2020.
- [204] Y. Zhu, J. Wong, A. Mandlekar, R. Martín-Martín, A. Joshi, S. Nasiriany, and Y. Zhu, “robosuite: A modular simulation framework and benchmark for robot learning,” in *arXiv preprint arXiv:2009.12293*, 2020.
- [205] E. Jang, A. Irpan, M. Khansari, D. Kappler, F. Ebert, C. Lynch, S. Levine, and C. Finn, “BC-Z: Zero-shot task generalization with robotic imitation learning,” in *Proceedings of the 5th Conference on Robot Learning*, pp. 991–1002, 2022.
- [206] D. Kalashnikov, J. Varley, Y. Chebotar, B. Swanson, R. Jonschkowski, C. Finn, S. Levine, and K. Hausman, “MT-Opt: Continuous multi-task robotic reinforcement learning at scale,” 2021.